

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Вятский государственный университет»
(ВятГУ)

УТВЕРЖДАЮ

Руководитель практики от Университета

(должность, ФИО)

МП

(Подпись)

Дата «__» _____ 202__ год

**ОТЧЕТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

ПМ.05 Проектирование и разработка информационных систем

Белых Максим Александрович

(Ф.И.О. обучающегося)

09.02.07 Информационные системы и программирование

(специальность)

Учебная группа

ИСПк-202-52-00

Место прохождения практики

ФГБОУ ВО «Вятский государственный университет»,

Колледж ВятГУ

(наименование организации, структурного подразделения организации)

Итоговая оценка:

Руководитель

практики от университета

(дата)

(подпись)

(Ф.И.О.)

Киров, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	2
1. СВЕДЕНИЯ О РАБОТЕ, ВЫПОЛНЕННОЙ В ПЕРИОД ПРОХОЖДЕНИЯ УЧЕБНОЙ ПРАКТИКИ.....	3
2. НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ.....	4
2.1. Выбор языка программирования	4
2.2. Используемые библиотеки Python.....	7
2.3. Выбор интегрированной среды разработки (IDE)	9
2.3.1. Причины выбора Visual Studio Code	9
2.4. Использование системы контроля версий Git и платформы GitHub.....	12
3. ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ.....	14
4. ОПИСАНИЕ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ... 15	15
4.1. Анализ предметной области.....	15
4.1.1. Анализ проблемы.....	16
4.1.2. Обзор аналогов.....	17
4.2. Разработка структуры приложения и алгоритмов функционирования	21
4.2.1. Разработка алгоритмов функционирования	21
4.2.2. Проектирование интерфейса	27
4.3. Разработка алгоритмов работы программы и программного обеспечения	34
4.3.1. Разработка программного обеспечения	34
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	41
ПРИЛОЖЕНИЕ А.....	43

ВВЕДЕНИЕ

Учебная практика ПМ.05 проходила на базе Колледжа ФГБОУ ВО «Вятский государственный университет» в период с 17.01.2026 г. по 20.06.2026 г. по субботам.

Цель практики: сформировать у обучающихся представление о процессе разработки прикладного программного обеспечения.

Задачи практики:

- дать представление о различных методах проектирования прикладного программного обеспечения;
- сформировать навыки написания документации, требуемой при разработке прикладного программного обеспечения;
- дать представление о различных инструментах, используемых при разработке прикладного программного обеспечения.

1. СВЕДЕНИЯ О РАБОТЕ, ВЫПОЛНЕННОЙ В ПЕРИОД ПРОХОЖДЕНИЯ УЧЕБНОЙ ПРАКТИКИ

В период с 17.01.2026 по 20.06.2026 при прохождении учебной практики ПМ.05 на базе Колледжа ФГОБУ ВО «Вятский государственный университет» был выполнен следующий перечень работ, представленный в таблице 1.

Таблица 1 – Сведения о работе, выполненной в период практики

Дата	Краткое содержание выполненных работ
17.01.2026	Ознакомление с правилами внутреннего трудового распорядка, охраны труда, техники безопасности, противопожарной безопасности, санитарно-эпидемиологическими правилами и гигиеническими нормативами, а также вводный инструктаж и инструктаж на рабочем месте
18.01.2026-31.01.2026	Постановка задачи
01.02.2026-13.02.2026	Настройка рабочего окружения
14.02.2026-27.02.2026	Анализ предметной области, постановка расширенного технического задания
28.02.2026-02.04.2026	Проектирование
03.04.2026-07.05.2026	Реализация программного кода
08.05.2026-28.05.2026	Тестирование и отладка полученного кода
29.05.2026-14.06.2026	Подготовка отчетной документации, включая детальное описание разработанных и использованных алгоритмов
18.06.2026-20.06.2026	Подготовка к сдаче и сдача промежуточной аттестации

(дата)

(подпись)

2. НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ

В ходе прохождения учебной практики использовался язык программирования Python, среда разработки Visual Studio Code или же более сокращенное название VS Code.



Рисунок 2.1 – Среда разработки Visual Studio Code

2.1. Выбор языка программирования

Выбор Python в качестве основного инструмента разработки менеджера задач обусловлен необходимостью баланса между функциональностью системы, скоростью реализации и долгосрочной поддержкой продукта. Решение базируется на следующих ключевых факторах:

- прежде всего, Python предоставляет разработчику мощный инструментарий для работы с форматами обмена данными, в частности JSON и XML, а также для проектирования графических интерфейсов. Безусловно, это позволяет организовать бесшовную передачу информации между модулями менеджера задач и создать интуитивно понятный интерфейс пользователя без существенных временных затрат на написание низкоуровневого кода;

– кроме того, использование пакетного менеджера `pip` совместно с изолированными виртуальными окружениями `venv` или `Poetry` позволяет подключать необходимые библиотеки одной командой. Подобный подход гарантирует стабильную воспроизводимость сборки менеджера задач на различных устройствах и значительно упрощает процесс масштабирования проекта. Следовательно, для добавления новой функции, например, интеграции с внешним календарем или ботом в различных мессенджерах, достаточно лишь интегрировать проверенное сообществом решение;

– в свою очередь, характерный для данного языка простой синтаксис существенно сокращает объем исходного кода. Это заметно ускоряет процесс прототипирования системы управления задачами и снижает порог входа для новых участников команды разработчиков [1];

– важно отметить, что поддержка парадигмы асинхронного программирования позволяет создавать отзывчивые интерфейсы и высокопроизводительные модули. Таким образом, менеджер задач обретает способность обрабатывать множество фоновых процессов, таких как своевременная отправка напоминаний, без блокировки основного потока приложения;

– статус Python как самого популярного языка по индексу ТЮВЕ за январь 2026 года гарантирует наличие актуальной документации и регулярных обновлений безопасности. Несомненно, наличие обширного сообщества способствует оперативному решению сложных технических задач, возникающих в ходе развития проекта [2].

Вместе с тем, для обеспечения объективности технического решения, необходимо учитывать недостатки Python, которые могут повлиять на эксплуатационные характеристики системы:

– в сравнении с компилируемыми языками, Python обладает меньшей скоростью выполнения из-за интерпретируемой природы и наличия Global Interpreter Lock (GIL) [5, 6]. Однако для задач Менеджера задач, не связанных с

тяжелыми математическими вычислениями, данное ограничение не является критическим;

- высокоуровневые структуры данных могут приводить к повышенному потреблению оперативной памяти [7]. Для нивелирования этого фактора при разработке планируется использовать оптимизированные типы данных и инструменты профилирования;

- с целью снижения риска возникновения ошибок в процессе исполнения, в проекте будет внедрено строгое использование аннотаций типов и многоуровневое автоматизированное тестирование [8].

Кроме того, Python является кроссплатформенным языком, что обеспечивает гибкость в развертывании и сопровождении разрабатываемого программного продукта.

Таким образом, выбор Python как основного инструмента разработки является обоснованным и рациональным с точки зрения как технической реализации проекта, так и его дальнейшей поддержки и развития.

2.2. Используемые библиотеки Python

Для реализации функционала менеджера задач используется набор библиотек, включающий как внешние зависимости, так и модули стандартной библиотеки Python.

Рассмотрим внешние зависимости в проекте:

- PySide6 (версия 6.10.1) выступает основным инструментом разработки графического интерфейса пользователя (GUI). Библиотека обеспечивает работу главного окна, диалоговых окон, системы сигналов и слотов, а также механизмов анимации и стилизации интерфейса через Qt Style Sheets;
- platformdirs применяется для кроссплатформенного определения путей к директориям конфигураций и данных пользователя в зависимости от операционной системы (Windows, Linux, macOS);
- shiboken6, PySide6_Essentials, PySide6_Addons используются как сопутствующие инфраструктурные компоненты, необходимые для корректного функционирования среды PySide6 и взаимодействия Python с C++ библиотеками Qt.

Дополнительно разберём используемые стандартные библиотеки Python:

- dataclasses применяется для создания структурированных моделей данных (задачи, категории, настройки), обеспечивая компактность кода и удобную сериализацию;
- json используется в качестве основного формата для локального хранения данных приложения и метаданных резервных копий;
- pathlib обеспечивает унифицированную работу с путями файловой системы, включая проверку существования файлов и создание необходимых директорий;
- datetime задействована для обработки временных меток, управления дедлайнами задач и формирования метаданных при создании архивов;

- `zipfile` и `tempfile` используются в модуле резервного копирования для создания сжатых архивов пользовательских данных и их безопасного извлечения через временные директории;
- `uuid` применяется для генерации уникальных идентификаторов сущностей, в нашем случае, задач и категорий внутри системы;
- `shutil` используется для выполнения операций копирования и перемещения файлов, в частности при импорте пользовательских иконок и восстановлении данных из бэкапа;
- `typing` обеспечивает статическую типизацию кода, повышая его читаемость и упрощая процесс разработки;
- `sys`, `platform`, `ctypes` используются в точке входа приложения для передачи аргументов командной строки и настройки специфичного поведения интерфейса под конкретные операционные системы.

2.3. Выбор интегрированной среды разработки (IDE)

VS Code (Visual Studio Code) является одной из самых популярных сред разработки. Опрос Stack Overflow Developer Survey 2025 показал, что VS Code занимает лидирующую позицию среди всех IDE [3]. Результаты опроса предоставлены на рисунке 2.2.

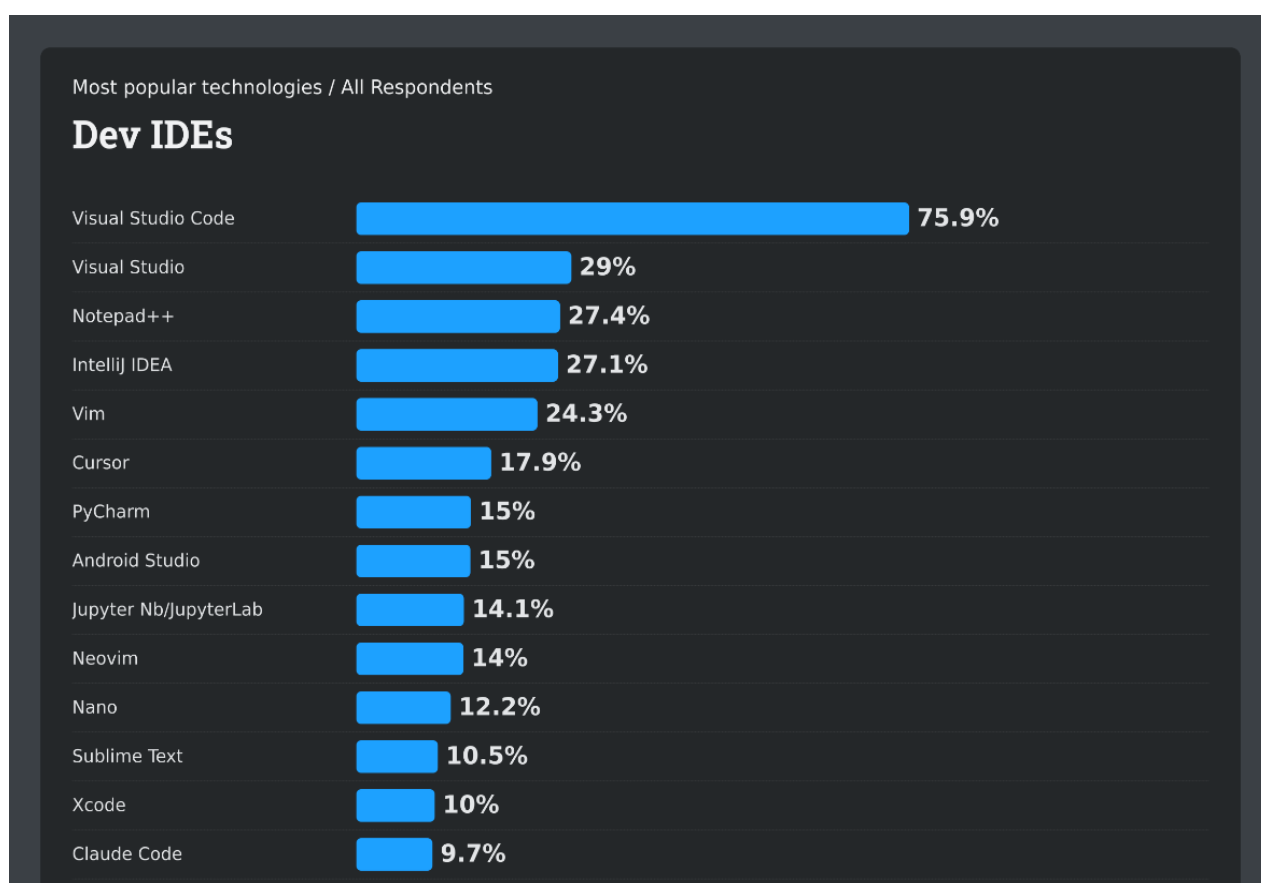


Рисунок 2.2 – Результаты опроса Stack Overflow Developer Survey 2025

2.3.1. Причины выбора Visual Studio Code

Широкое распространение Visual Studio Code в профессиональной среде обусловлено не столько тенденциями рынка, сколько совокупностью эффективных архитектурных решений и широких функциональных возможностей. Данный программный продукт удовлетворяет техническим требованиям специалистов различного уровня квалификации – от начинающих

разработчиков до ведущих инженеров. Ниже представлен подробный анализ ключевых факторов, определяющих этот выбор. Рассмотрим подробнее каждую из причин:

- согласно результатам опроса Stack Overflow, Visual Studio Code (VS Code) является самым популярным редактором кода среди разработчиков [3]. Благодаря широкой поддержке плагинов, включая те, что предназначены для Python, он активно используется как новичками, так и опытными программистами. Это делает его удобным инструментом для разработки на Python благодаря гибкости, простоте настройки и большому сообществу пользователей;

- VS Code поддерживает не только Python, но и множество других языков программирования и технологий, включая JavaScript, C++, Java и другие. Это позволяет использовать одну среду для разработки различных проектов, что особенно важно для многозадачных разработчиков [4];

- VS Code позволяет легко интегрировать нужные расширения и плагины для работы с фреймворками, базами данных, системами контроля версий и даже для анализа кода. В отличие от других IDE, VS Code предоставляет пользователю возможность создать рабочее пространство, идеально подходящее под его нужды [4];

- благодаря огромному числу пользователей и активному сообществу, в VS Code всегда можно найти решения для большинства проблем и настроить среду под любые задачи. Поддержка множества языков и фреймворков делает его универсальным инструментом для решения самых разных задач [3];

- VS Code имеет чистый и минималистичный интерфейс, который легко освоить. Несмотря на свою простоту, IDE включает в себя мощные инструменты для работы с кодом, такие как автодополнение, рефакторинг, а также интеграцию с Git [4].

В качестве основной среды разработки была выбрана VS Code. Выбор обусловлен её многофункциональностью, гибкостью и поддержкой широкого

стека инструментов, что обеспечивает эффективную работу как с Python, так и со смежными технологиями. Интерфейс показан на рисунке 2.3.

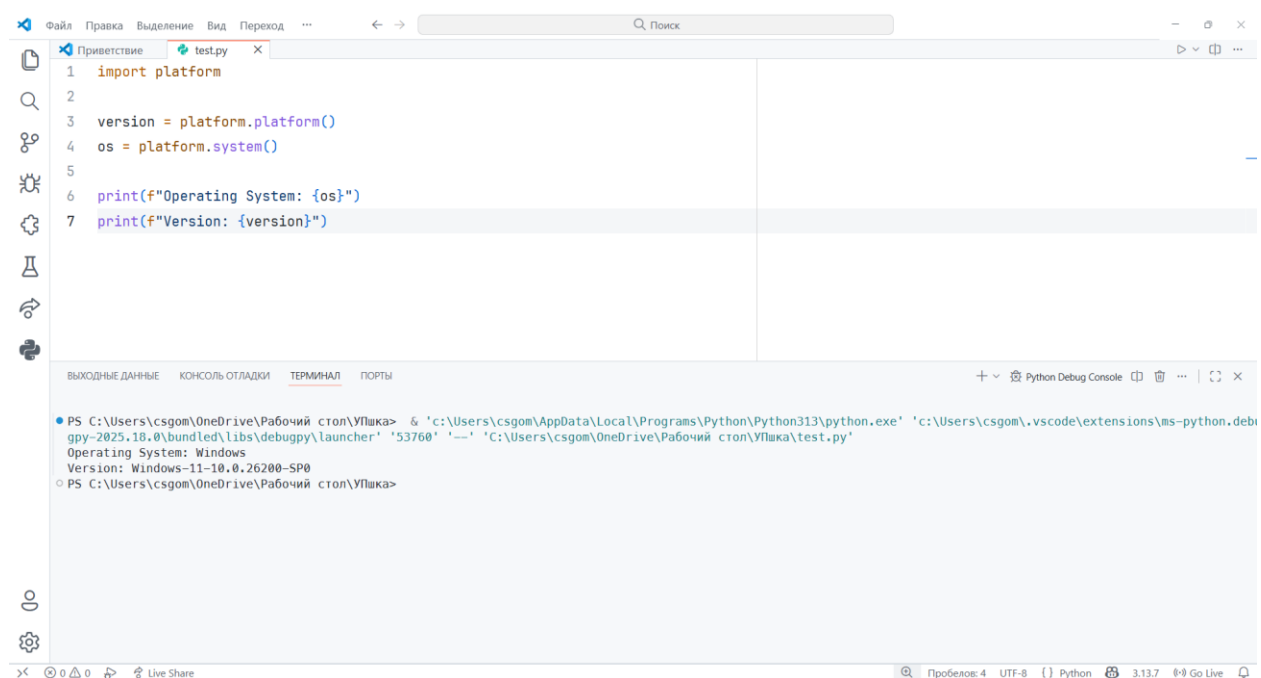


Рисунок 2.3 – Интерфейс среды разработки

2.4. Использование системы контроля версий Git и платформы GitHub

Для управления версиями программного кода в соответствии с заданием использована распределенная система контроля версий Git и облачная платформа GitHub. Ссылка на профиль: <https://github.com/MegaM15C>. Внешний вид личного профиля продемонстрирован на рисунке 2.4.

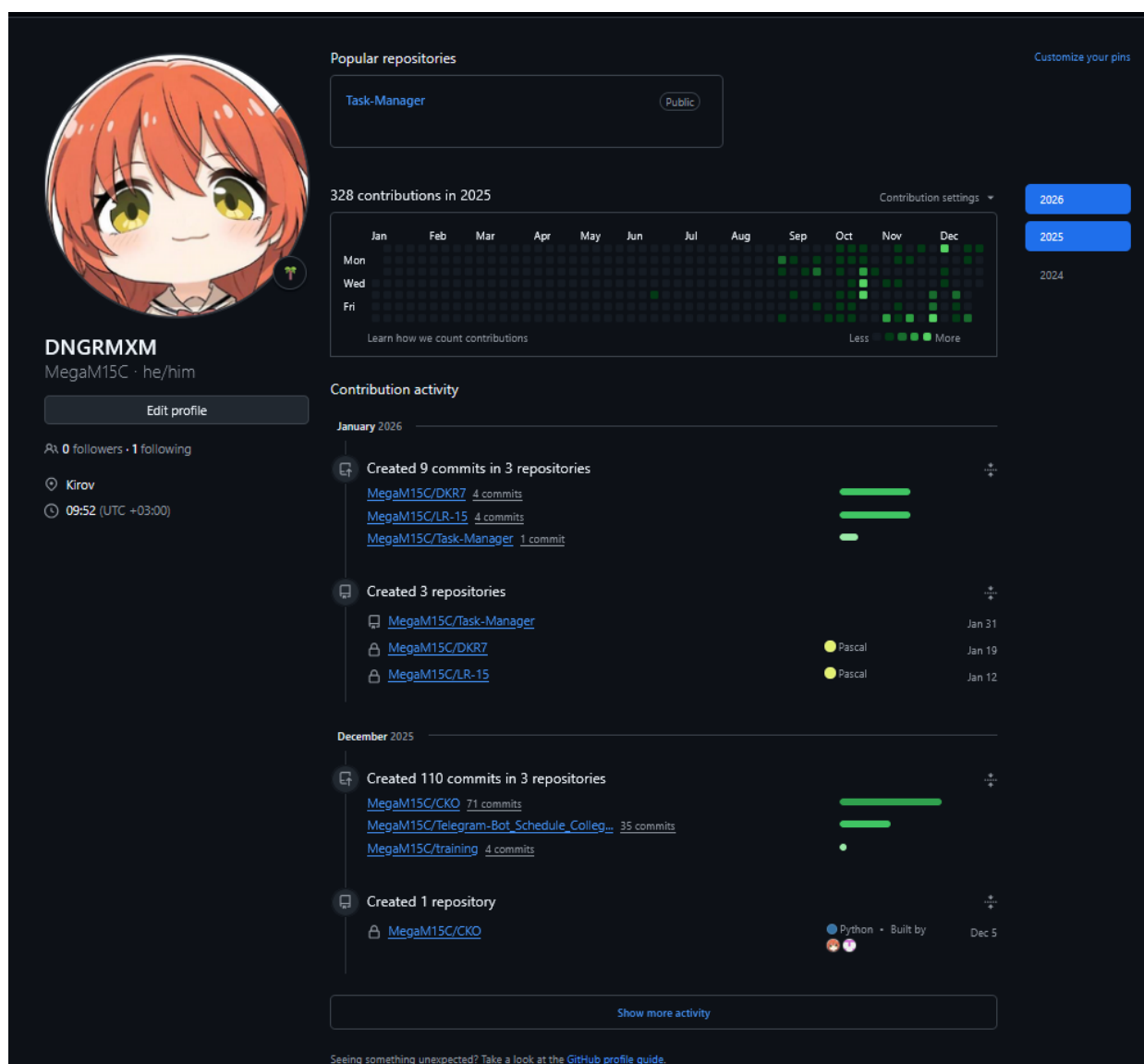


Рисунок 2.4 – Личный профиль в GitHub

Для обеспечения доступности исходного кода и фиксации этапов разработки в системе GitHub создан публичный репозиторий. Процесс создания

включал определение названия проекта, настройку параметров приватности и выбор типа лицензии. Ссылка на репозиторий проекта: <https://github.com/MegaM15C/Task-Manager>. Факт создания репозитория представлен на рисунке 2.5.

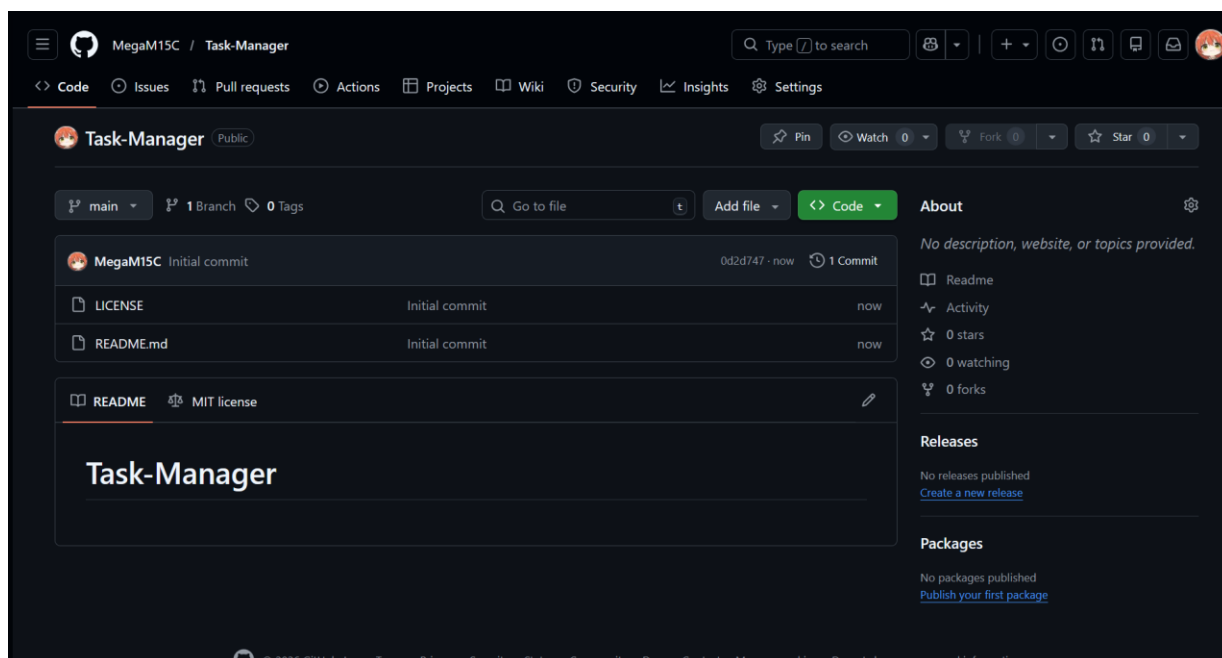


Рисунок 2.5 – Репозиторий на GitHub

3. ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

В соответствии с индивидуальным заданием на учебную практику необходимо:

- пройти обязательный инструктаж по технике безопасности и ознакомиться с правилами внутреннего распорядка в организации;
- сформулировать требования к программному продукту и выполнить постановку задачи;
- подготовить рабочее окружение, включая установку и настройку необходимых средств разработки;
- провести детальный анализ предметной области и составить расширенное техническое задание;
- выполнить проектирование архитектуры приложения и интерфейса пользователя;
- осуществить написание программного кода согласно разработанному проекту;
- провести тестирование функционала и отладку выявленных ошибок в коде;
- оформить отчетную документацию с описанием реализованных алгоритмов;
- подготовить материалы к промежуточной аттестации и защитить результаты практики.

4. ОПИСАНИЕ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1. Анализ предметной области

Задача – это формализованное описание действия или набора действий, выполнение которых необходимо для достижения определённого результата в заданный период времени [9].

Управление задачами – это процесс планирования, организации, контроля и отслеживания задач с целью эффективного достижения поставленных целей и результатов [10].

В современных условиях цифровизации управленческих процессов менеджер задач выступает в качестве высокотехнологичного инструментария, обеспечивающего координацию ресурсов. Ключевым аспектом анализа данной области является декомпозиция жизненного цикла задачи, который включает в себя следующие этапы: инициирование, планирование, исполнение, мониторинг и завершение [11].

Эффективность систем управления задачами определяется их способностью поддерживать методологии организации труда. К наиболее востребованным относятся:

- Agile-подходы (Scrum, Kanban) ориентированы на итеративную разработку и визуализацию потока работ;
- метод «Водопада» (Waterfall) предполагает строгую последовательность этапов;
- метод GTD (Getting Things Done) фокусируется на разгрузке оперативной памяти пользователя путем фиксации всех входящих задач во внешней системе [12].

Особое внимание при разработке менеджера приложений уделяется архитектуре данных и функциональным возможностям, таким как приоритезация (например, с использованием матрицы Эйзенхауэра), установка

дедлайнов, делегирование полномочий и нотификация исполнителей [13]. Анализ рынка существующих решений показывает тенденцию к интеграции менеджеров задач с облачными сервисами и системами совместной работы, что обуславливает необходимость реализации кроссплатформенности и синхронизации данных в реальном времени [14].

4.1.1. Анализ проблемы

Одной из центральных проблем является информационная перегруженность и «эффект накопления». В классических системах управления задачами по мере роста объема данных происходит снижение контролируемости списка (т. н. «бесконечный бэклог»). Отсутствие механизмов автоматической архивации или интеллектуальной скрытости неактуальных пунктов приводит к рассеиванию внимания пользователя. Психологические исследования подтверждают, что визуальный шум в интерфейсе напрямую коррелирует с ростом уровня стресса и прокрастинации [15].

Вторая группа проблем связана с недостаточной гибкостью моделей приоритезации. Большинство систем предлагают лишь дискретные уровни важности (например, «высокий», «средний», «низкий»), что не позволяет адекватно оценить задачу в динамике. Проблемными зонами здесь являются:

- статичность приоритетов: отсутствие автоматического повышения значимости задачи по мере приближения крайнего срока (дедлайна);
- игнорирование контекста выполнения: системы редко учитывают требуемый приоритет задачи при формировании плана на день, что ведет к некорректному распределению ресурсов исполнителя [16].

Также существенной проблемой является отсутствие качественной аналитики и обратной связи. Большинство менеджеров задач функционируют как пассивные хранилища данных, не предоставляя пользователю инструментов для ретроспективного анализа его эффективности. Без механизмов визуализации фактических трудозатрат в сравнении с

планируемыми невозможно выявить «поглотители времени» и скорректировать личные или командные регламенты работы [17].

Таким образом, актуальность разработки нового менеджера задач обусловлена необходимостью создания инструмента, который минимизирует когнитивную нагрузку через адаптивный интерфейс и предоставляет развитую систему аналитики для управления личной эффективностью.

4.1.2. Обзор аналогов

Современный рынок систем управления задачами представлен широким спектром программных продуктов, которые можно разделить на три сегмента: универсальные планировщики, инструменты для гибкой разработки и минималистичные персональные менеджеры.

Первым аналогом представлен Jira, который является эталонным решением для управления сложными проектами в рамках методологии Agile. С технической точки зрения система базируется на мощном движке управления рабочими процессами, позволяющем кастомизировать каждый этап жизненного цикла задачи.

Рассмотрим преимущества:

- глубокая аналитика (диаграммы сгорания задач, контрольные карты);
- высокая степень автоматизации.

Недостатки:

- избыточная сложность интерфейса;
- высокая кривая обучения;
- значительные требования к системным ресурсам;
- полноценное использование возможно только после покупки лицензии.

Jira является избыточной для персонального использования и малых групп, так как временные затраты на администрирование системы часто

превышают выгоду от её использования в простых проектах. Интерфейс Jira продемонстрирован на рисунке 4.1.

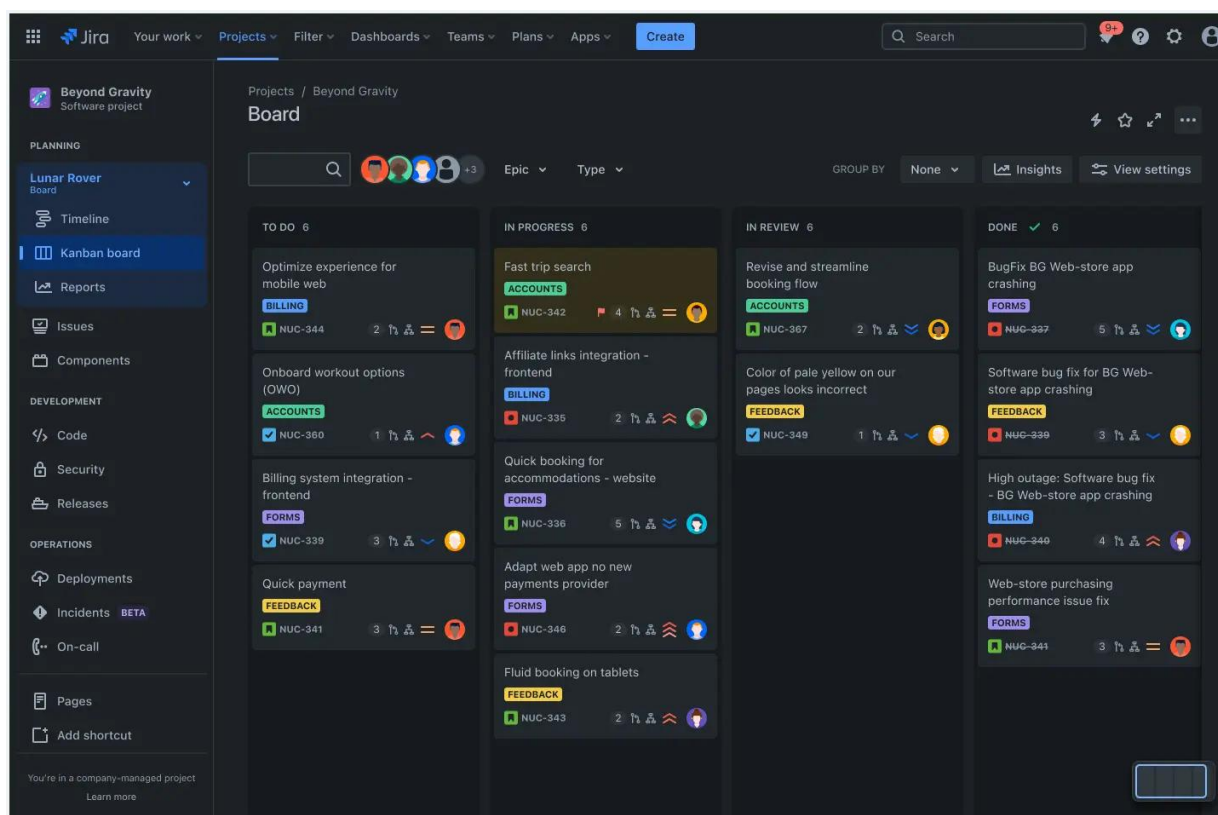


Рисунок 4.1 – Интерфейс Jira

Вторым значимым аналогом, заслуживающим детального рассмотрения в рамках данной работы, является программный продукт Trello. Данная система представляет собой популярный инструмент, реализующий визуальную парадигму Kanban-досок, которая получила широкое распространение в современной практике проектного менеджмента. В основе технической реализации сервиса лежит предельно упрощенная, но эффективная объектная модель, выстроенная по принципу: «доска – список – карточка». Такая иерархия позволяет пользователю моделировать рабочие процессы любой направленности, трансформируя абстрактные задачи в осязаемые визуальные элементы.

Преимущества:

- интуитивно понятный интерфейс;

- низкий порог вхождения;
- гибкость визуализации.

Рассмотрим недостатки:

- ограниченные возможности для работы с иерархическим структурами задач;
- отсутствие встроенных инструментов для глубокого анализа временных затрат без сторонних расширений;
- система не предоставляет встроенных механизмов для автоматизированного учета временных затрат и построения отчетов о производительности (например, диаграмм Ганта) без интеграции платных сторонних расширений («Power-Ups»).

Trello демонстрирует высокую эффективность в задачах визуализации текущего. Однако отсутствие жесткой логической структуры данных и дефицит встроенных инструментов аналитики продуктивности делают данную систему малоприспособленной для дисциплинированного управления долгосрочными стратегическими целями и сложными иерархическими проектами, требующими точного соблюдения временных регламентов. Интерфейс представлен на рисунке 4.2.

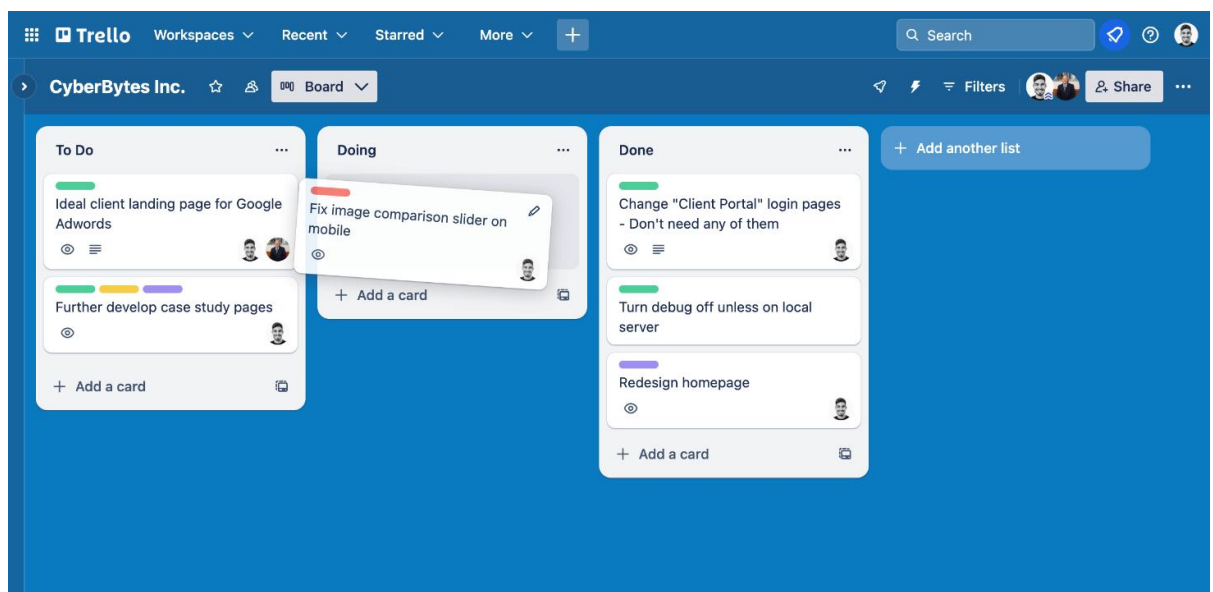


Рисунок 4.2 – Интерфейс Trello

Третьим и последним аналогом является Microsoft To Do. Этот аналог является представителем сегмента персональных менеджеров. Ориентирован на быстрое управление списками дел с использованием интеллектуального алгоритма формирования «Моего дня».

Преимущества данного аналога:

- высокая скорость работы интерфейса;
- минималистичный дизайн;
- эффективная работа с напоминаниями.

Недостатки:

- отсутствие оценки задач по сложности и трудозатратам;
- ограниченный функционал для работы с многоуровневыми проектами;
- отсутствие инструментов для глубокого ретроспективного анализа эффективности.

Данное решение идеально подходит для бытовых списков покупок или простых напоминаний, но не обеспечивает пользователя инструментами приоритезации по сложности или аналитическим базисом для профессионального роста. Визуальная часть Microsoft To Do можно увидеть на рисунке 4.3.

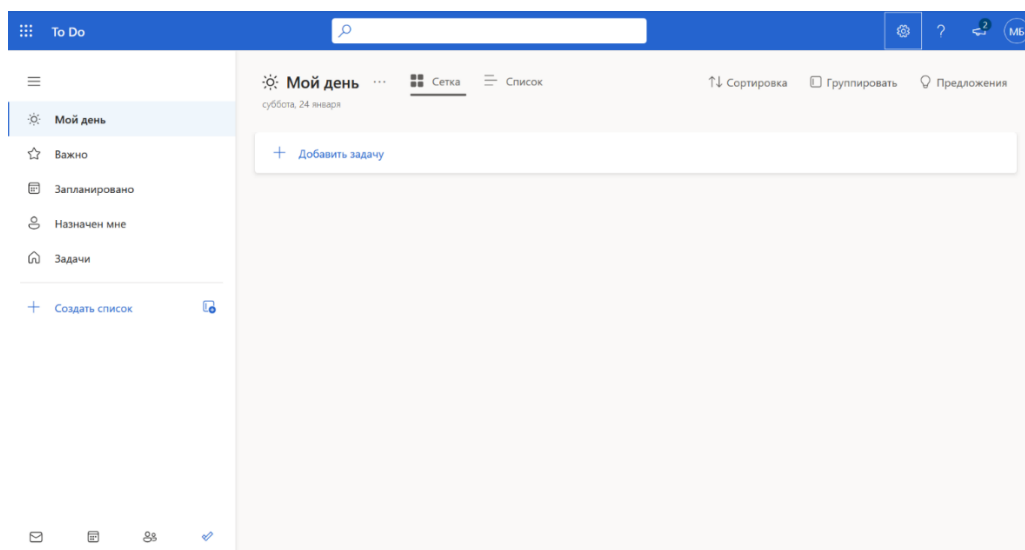


Рисунок 4.3 – Интерфейс Microsoft To Do

Проведенный сравнительный анализ ключевых аналогов (Jira, Trello, Microsoft To Do) показал, что современный рынок систем управления задачами характеризуется функциональной полярностью. С одной стороны, представлены избыточно сложные корпоративные инструменты, требующие длительного обучения и значительных системных ресурсов. С другой стороны, существуют упрощенные планировщики, функционала которых недостаточно для ведения многоуровневых проектов и дисциплинированного соблюдения временных регламентов.

Выявленный дефицит доступных инструментов для автоматизированного контроля сроков и интеллектуальной приоритезации задач подтверждает актуальность разработки нового менеджера задач. Проектируемая система призвана стать «золотой серединой», сочетая в себе интуитивно понятный интерфейс с гибкой системой параметров (дедлайны, уровни важности, категории). Это позволит пользователю не только фиксировать текущие дела, но и эффективно распределять личные ресурсы, минимизируя когнитивную нагрузку при принятии решений и планировании рабочего дня.

4.2. Разработка структуры приложения и алгоритмов функционирования

4.2.1. Разработка алгоритмов функционирования

Алгоритм работы программы «Менеджер задач» представлен на рисунке 4.2.1. Данная схема описывает процесс инициализации системы, обработки данных и циклического обновления интерфейса.

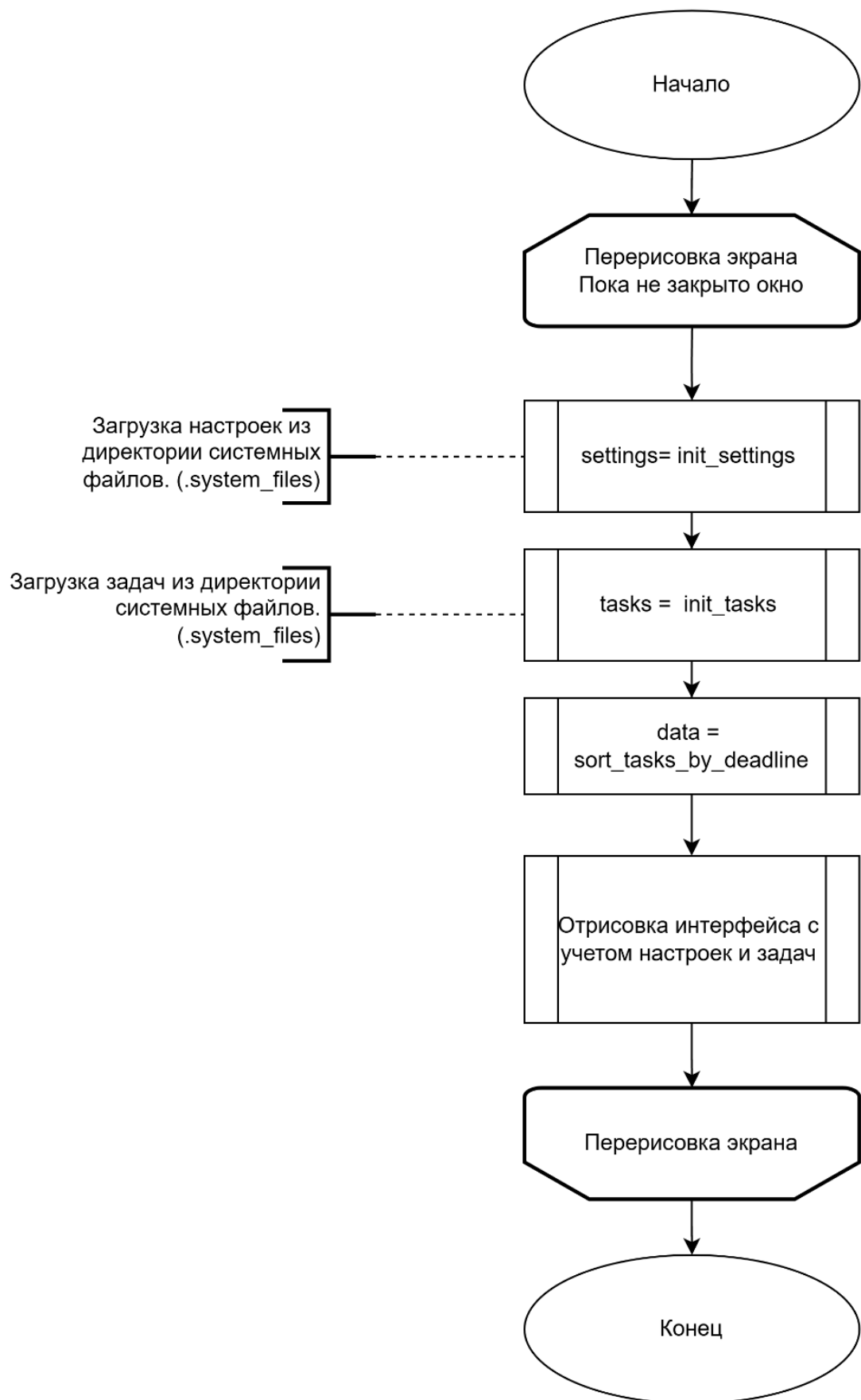


Рисунок 4.2.1 – Алгоритм работы разрабатываемого приложения

Алгоритм включает в себя вспомогательные алгоритмы, рассмотрим каждый из них.

Первым рассмотрим алгоритм инициализации задач. Алгоритм представлен на рисунке 4.2.2.

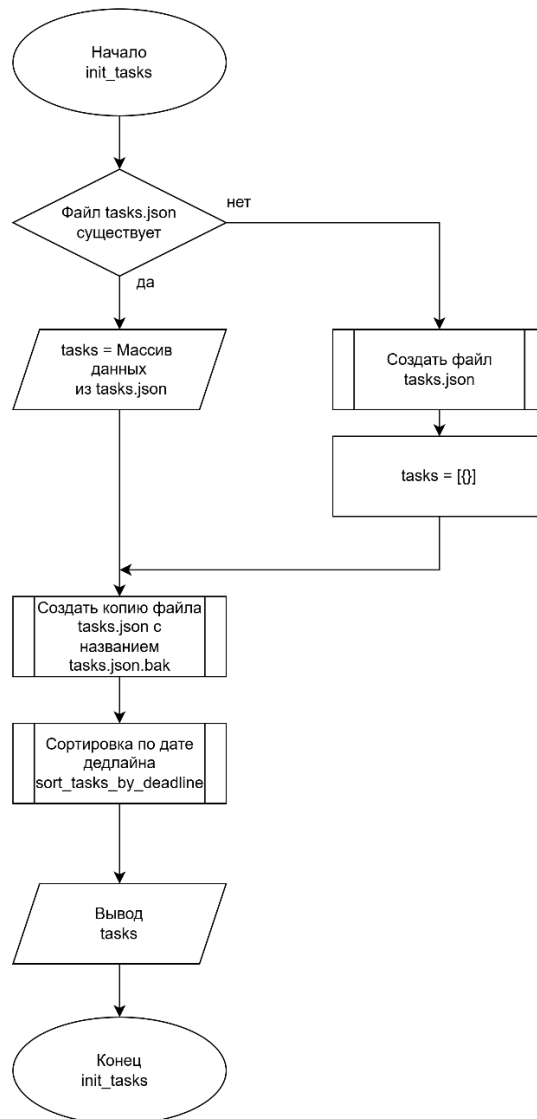


Рисунок 4.2.2 – Алгоритм инициализации задач

Алгоритм инициализации задач (init_tasks) отвечает за корректную загрузку списка дел при запуске приложения:

- проверка файла: система проверяет наличие файла tasks.json;

- загрузка или создание: если файл существует, данные считываются в массив. Если нет – создается новый пустой файл и массив;
- резервное копирование: для безопасности создается копия текущего файла с расширением .bak;
- обработка: перед выводом данные проходят через функцию сортировки по дедлайнам;

Вторым вспомогательным алгоритмом является инициализация настроек. Представлен на рисунке 4.2.3.

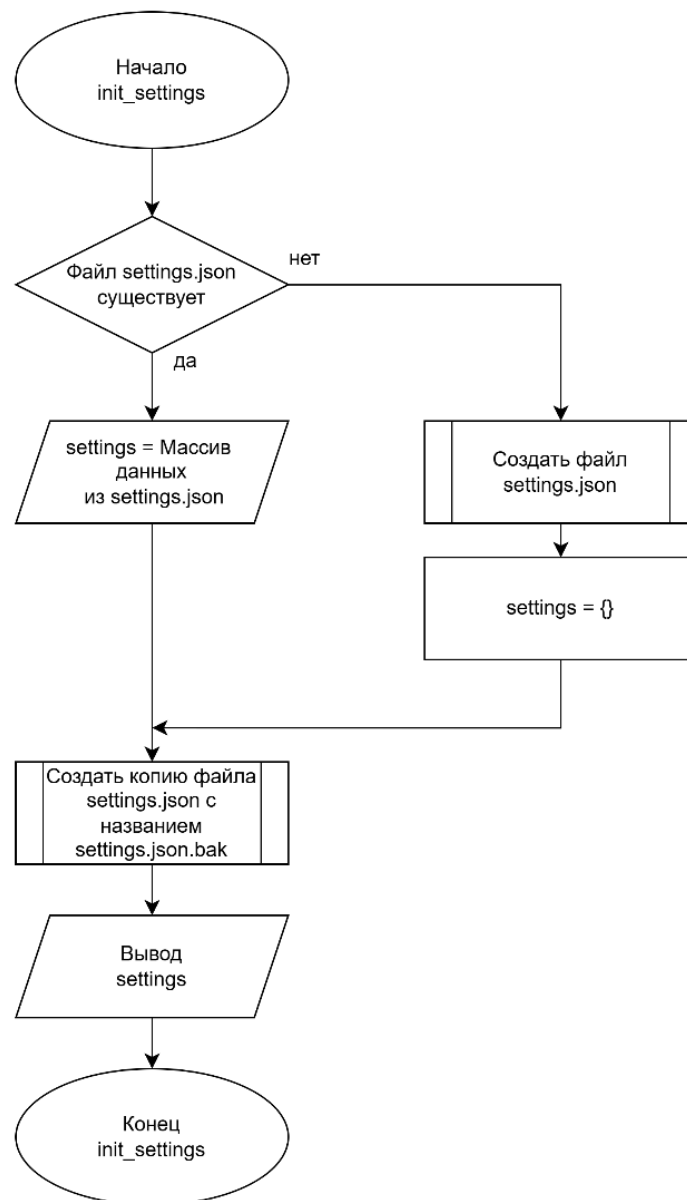


Рисунок 4.2.3 – Алгоритм инициализации настроек

Алгоритм инициализации настроек (`init_settings`) является процессом подготовки интерфейса и пользовательских предпочтений:

- работа с конфигом: аналогично задачам, проверяется файл `settings.json`;
- отказоустойчивость: при отсутствии файла настроек система генерирует стандартный пустой объект настроек;
- безопасность: создается резервная копия настроек (`settings.json.bak`);
- результат: массив параметров (тема, шрифт, цвета) передается для отрисовки интерфейса.

Последним вспомогательным алгоритмом является сортировка задач по дедлайнам. Представлен на рисунке 4.2.4.

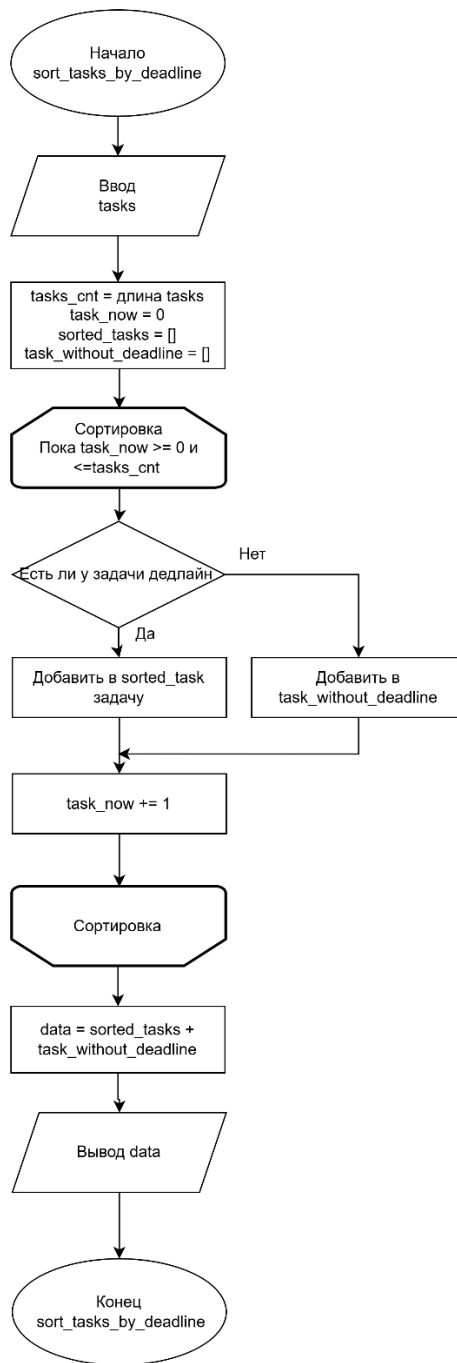


Рисунок 4.2.4 – Алгоритм сортировки задач по дедлайнам

Алгоритм сортировки по дедлайнам (sort_tasks_by_deadline) – это вспомогательный процесс, который упорядочивает список задач:

- разделение: в цикле программа проверяет каждую задачу на наличие установленной даты дедлайна;
- группировка: задачи с датами попадают в список sorted_tasks, а задачи без дат – в task_without_deadline;

- слияние: в финале оба списка объединяются в один массив data, где сначала идут срочные задачи, а затем все остальные.

Поэтапная работа главной программы:

- запуск и инициализация: при старте приложение входит в основной цикл «Перерисовка экрана», который остается активным до тех пор, пока пользователь не закроет окно;

- загрузка конфигураций («init_settings»): система обращается к директории системных файлов (формат «.system_files») для извлечения пользовательских настроек (цветовая схема, параметры уведомлений и т.д.).

- инициализация данных («init_tasks»): программа считывает список текущих задач из системного хранилища. Это позволяет сохранять прогресс между сессиями;

- обработка и сортировка («sort_tasks_by_deadline»): полученный массив данных проходит через процедуру автоматической сортировки. Задачи упорядочиваются по приоритету и близости даты дедлайна для удобства отображения;

- отрисовка интерфейса: на основе загруженных настроек и отсортированного списка задач формируется визуальное представление;

- цикл обновления: программа постоянно отслеживает изменения. Если пользователь вносит правки, например, добавляет задачу или меняет её статус, цикл перерисовки актуализирует данные на экране;

- завершение работы: при закрытии окна пользователем выполнение алгоритма переходит в блок «Конец», завершая все процессы и сохраняя текущее состояние.

4.2.2. Проектирование интерфейса

Главное окно приложения спроектировано с учетом принципов эргономичности и интуитивно понятной навигации. Интерфейс разделен на функциональную боковую панель и основную рабочую область. В левой части экрана расположены фильтры по категориям и системные настройки, включая

переключатель темной темы и кнопку добавления новых категорий. Верхняя панель содержит заголовок текущего раздела и кнопку «Добавить задачу» для быстрого создания новых записей.

Центральную часть окна занимает динамический список задач, представленный в виде карточек с информацией о названии дела, установленном сроке и присвоенных тегах приоритетности. В нижней части экрана предусмотрена индикация состояния системы, информирующая о процессе загрузки или синхронизации данных. Интерфейс главного окна представлен на рисунке 4.2.5.

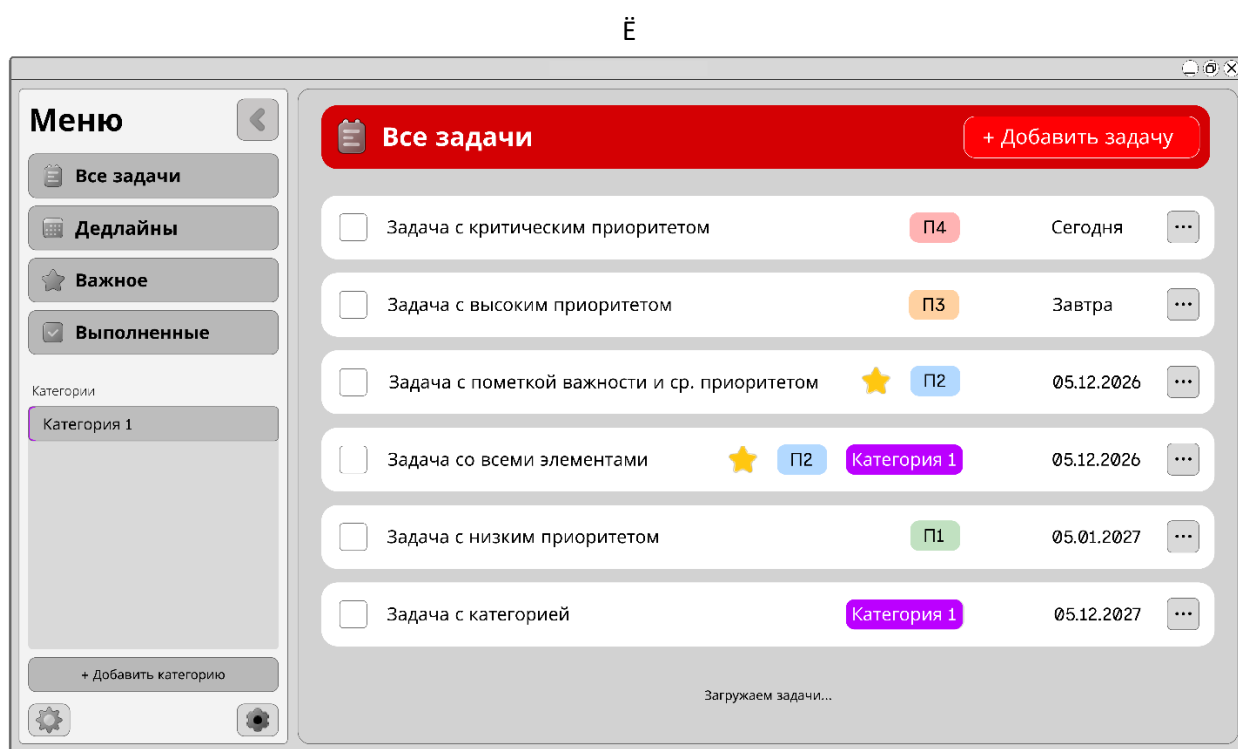


Рисунок 4.2.5 – Интерфейс главного окна

Окно настроек приложения реализовано в виде модального интерфейса, который позволяет пользователю адаптировать визуальную среду под индивидуальные предпочтения. Основные параметры сгруппированы в три логических блока: выбор акцентного цвета для элементов выделения, переключение основной темы оформления (светлая/темная) и настройка типографики через выбор системного шрифта. Использование выпадающих

списков и интерактивных кнопок выбора цвета обеспечивает мгновенный предпросмотр изменений, не перегружая основной экран лишними элементами управления.

Дизайн окна выполнен в лаконичном стиле с мягким затемнением заднего фона, что фокусирует внимание пользователя на конфигурации системы. Заголовки параметров сопровождаются краткими пояснительными подписями, облегчая взаимодействие с интерфейсом даже для начинающих пользователей. Закрытие окна осуществляется с помощью стандартной кнопки-иконки в верхнем правом углу, при этом выбранные настройки сохраняются в конфигурационный файл приложения автоматически или после подтверждения, согласно алгоритму работы с системными файлами. Интерфейс модального окна настроек представлен на рисунке 4.2.6.

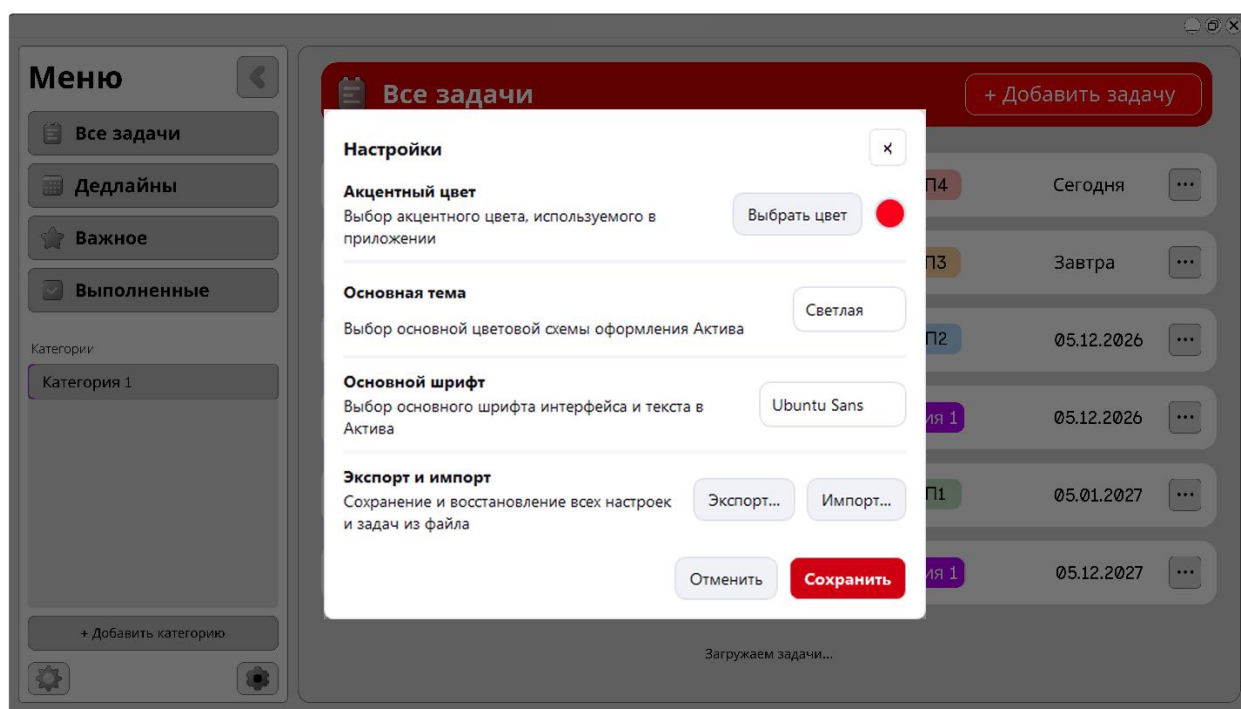


Рисунок 4.2.6 – Интерфейс модального окна настроек

Окно создания категории в приложении спроектировано как компактный мастер настройки, позволяющий персонализировать группировку задач. Интерфейс включает три ключевых параметра: текстовое поле для ввода

уникального названия категории, инструмент выбора акцентного цвета для визуальной идентификации тега в общем списке и функционал загрузки пользовательской иконки. Такой подход обеспечивает гибкость системы, позволяя пользователю визуально отделять рабочие процессы от личных дел с помощью уникальных графических маркеров.

Интерфейс модального окна сохраняет общую стилистику приложения, используя закругленные формы и понятную иерархию элементов. Для завершения процесса предусмотрены кнопки «Создать» и «Отменить», выделенные контрастным цветом для предотвращения случайных действий. Новая категория после подтверждения мгновенно интегрируется в боковую панель управления, становясь доступной для назначения новым или уже существующим задачам, что позволяет динамически расширять структуру планировщика под текущие нужды пользователя. Интерфейс представлен на рисунке 4.2.7.

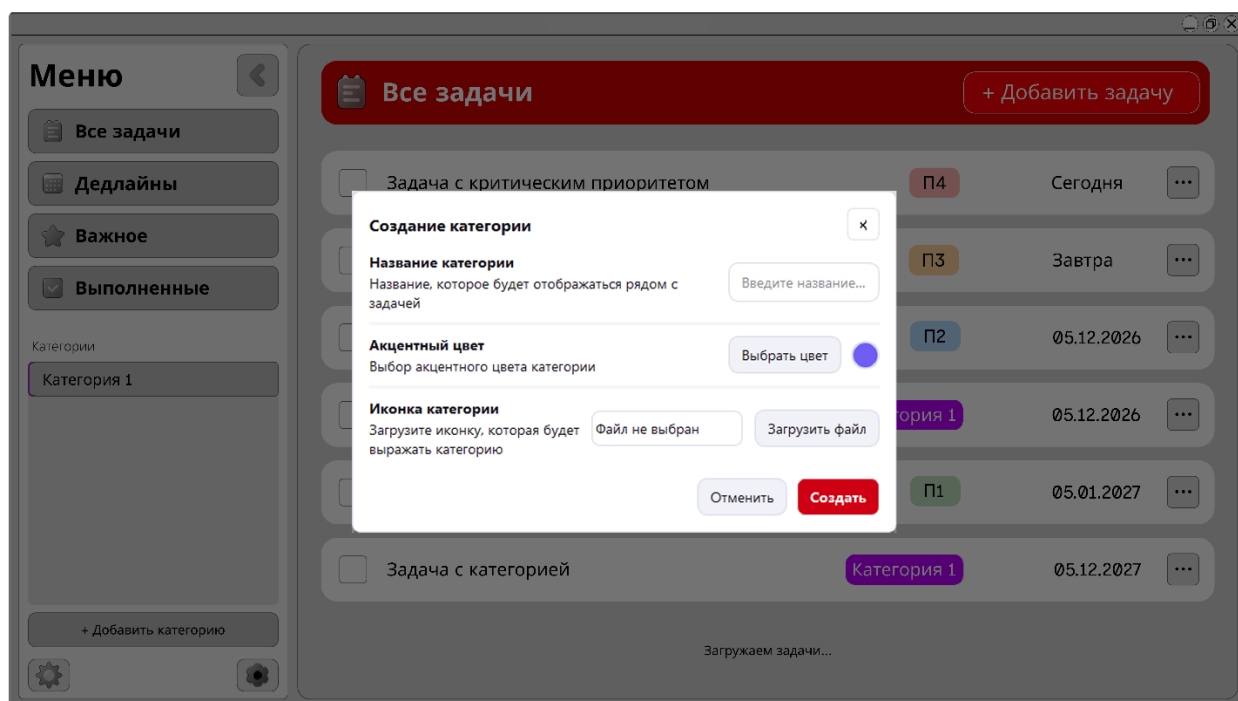


Рисунок 4.2.7 – Интерфейс модального окна создания категории задач

Окно добавления задачи в приложении представляет собой лаконичную форму, предназначенную для быстрого планирования дел с минимальными временными затратами. Пользователю предлагается заполнить пять основных полей: ввести краткое название, отражающее суть поручения, выбрать одну из существующих категорий из выпадающего списка, установить дату дедлайна с помощью интерактивного календаря, установить приоритет, например, высокий, и, опционально, установить флажок важная задача. Наличие подсказок под каждым заголовком помогает пользователю корректно заполнить данные, обеспечивая структурированность будущего списка задач.

Визуальное оформление модального окна соответствует общему дизайну коду системы: светлая подложка с мягкими тенями контрастирует с заблокированным основным интерфейсом, фокусируя внимание на процессе ввода. Завершение операции осуществляется нажатием кнопки «Создать», после чего задача автоматически сортируется в общем списке согласно установленному дедлайну. Возможность мгновенной отмены через соответствующую кнопку закрытия предотвращает создание ошибочных или дублирующих записей в базе данных приложения. Интерфейс создания задачи показан на рисунке 4.2.8.

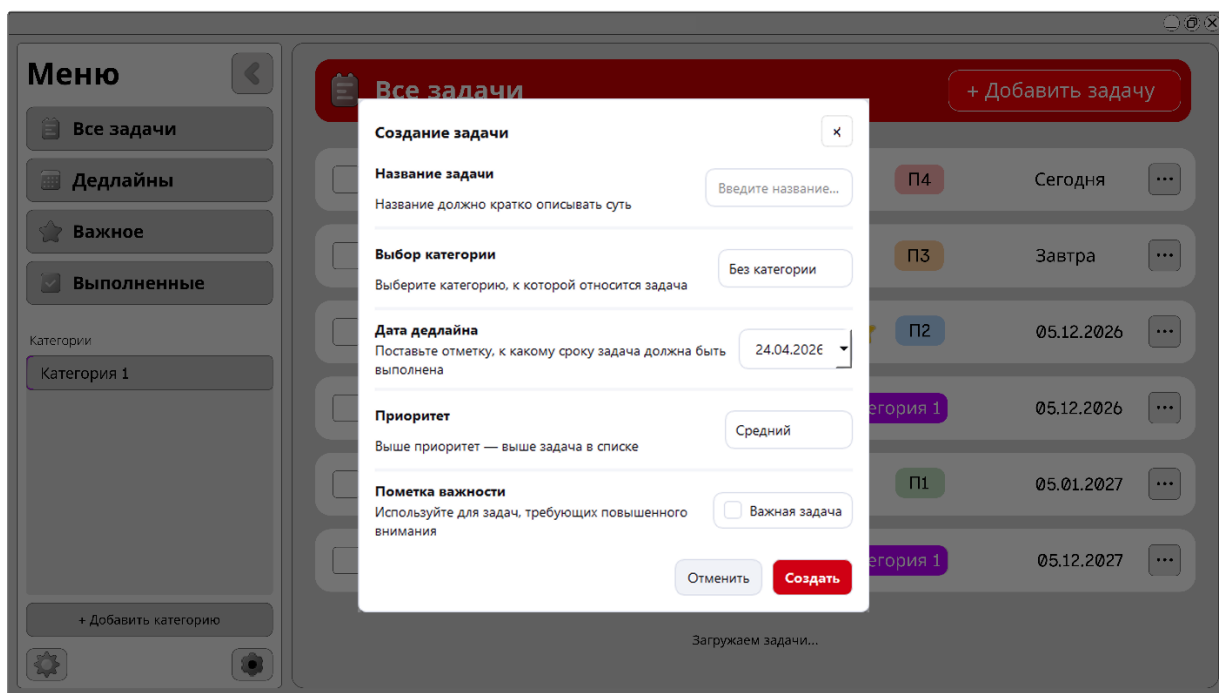


Рисунок 4.2.8 – Интерфейс создания задачи

Вывод по разделу:

В ходе выполнения работы была спроектирована и реализована программная система управления задачами, ориентированная на повышение эффективности персонального планирования. В рамках разработки была решена задача создания интуитивно понятного интерфейса, обеспечивающего быстрый доступ к ключевым функциям системы без избыточной сложности.

На основе анализа предметной области были достигнуты следующие результаты:

- разработаны алгоритмы функционирования, обеспечивающие корректную инициализацию пользовательских настроек, загрузку данных из локальных хранилищ и динамическую сортировку задач по приоритетам и срокам исполнения;
- спроектирован интерфейс пользователя, включающий главное рабочее окно с цветовой индикацией статусов, а также систему модальных окон для гибкой настройки приложения и управления категориями;

— реализована логика управления данными, позволяющая пользователю не только структурировать задачи по тематическим группам, но и адаптировать визуальную среду (темы, шрифты, акцентные цвета) под индивидуальные нужды.

Разработанный менеджер задач представляет собой законченный инструмент, который сочетает в себе функциональность классического планировщика с возможностью глубокой персонализации интерфейса. Созданная архитектура приложения позволяет в дальнейшем масштабировать систему, добавляя модули облачной синхронизации и расширенной аналитики продуктивности.

4.3. Разработка алгоритмов работы программы и программного обеспечения

В данном разделе произведена разработка архитектуры и алгоритмов функционирования приложения, спроектирован пользовательский интерфейс и описана реализация ключевых модулей системы.

4.3.1. Разработка программного обеспечения

Программное обеспечение разработано на языке Python 3 с использованием кроссплатформенного фреймворка PySide6 (Qt for Python). В соответствии с требованиями к надежности, хранение данных реализовано локально в формате JSON с применением алгоритмов атомарной записи.

Архитектура приложения разделена на логические уровни: ядро (модели и репозитории), интерфейс (виджеты и стили) и утилиты. Ключевым элементом обработки данных является класс `TasksRepository`, отвечающий за хранение задач. Для оптимизации производительности при работе с большим количеством записей реализовано постраничное хранение данных.

Фрагмент исходного кода метода `add` класса `TasksRepository`, отвечающего за создание новой задачи с учетом пагинации, представлен на рисунке 4.3.1.

```

def add(
    self,
    title: str,
    *,
    category_id: Optional[str],
    due: Optional[date],
    priority: int = 0,
    important: bool = False,
) -> Task:
    """Создаёт новую задачу и добавляет её в последнюю страницу.

    Если последняя страница заполнена (page_size), создаётся новый файл.
    """
    task = Task(
        id=uuid.uuid4().hex,
        title=title.strip(),
        category_id=category_id,
        due=due,
        priority=int(priority),
        important=bool(important),
        done=False,
        done_at=None,
    )

    files = self._page_files()
    if not files:
        path = self._paths.tasks_pages_dir / "page_0001.json"
        write_json_atomic(path, [_task_to_json(task)])
        return task

    last_path = files[-1]
    raw = read_json(last_path, default=[])
    if not isinstance(raw, list):
        raw = []

    if len(raw) >= self._page_size:
        new_path = self._next_page_path()
        write_json_atomic(new_path, [_task_to_json(task)])
        return task

    raw.append(_task_to_json(task))
    write_json_atomic(last_path, raw)
    return task

```

Рисунок 4.3.1 – Исходный код функции add

Для обеспечения динамического изменения внешнего вида приложения разработана система токенизации тем. Класс ThemeTokens и функция tokens_from_settings позволяют генерировать цветовую палитру в зависимости

от выбранной пользователем темы (светлая или тёмная). Исходный код функции генерации токенов представлен на рисунке 4.3.2.

```
def tokens_from_settings(s: Settings) -> ThemeTokens:
    """Создаёт набор цветовых токенов из объекта Settings."""
    if s.theme == "light":
        return ThemeTokens(
            accent=s.accent_color,
            bg="#F5F6FA",
            surface="FFFFFF",
            surface_2="#F0F2F7",
            text="#111318",
            text_muted="#5B6270",
            border="#D9DDEA",
        )
    return ThemeTokens(
        accent=s.accent_color,
        bg="#0F1116",
        surface="#151824",
        surface_2="#1B2030",
        text="#E9ECF5",
        text_muted="#A7AFC3",
        border="#2A3147",
    )
```

Рисунок 4.3.2 – Исходный код функции tokens_from_settings

Основным классом графического интерфейса является MainWindow. Он агрегирует в себе репозитории для работы с данными и управляет отображением списка задач. В классе реализована фильтрация и сортировка задач «на лету» с помощью метода `_compute_view_tasks`.

Диаграмма классов, отображающая взаимосвязи между основными модулями системы, представлена на рисунке 4.3.3.

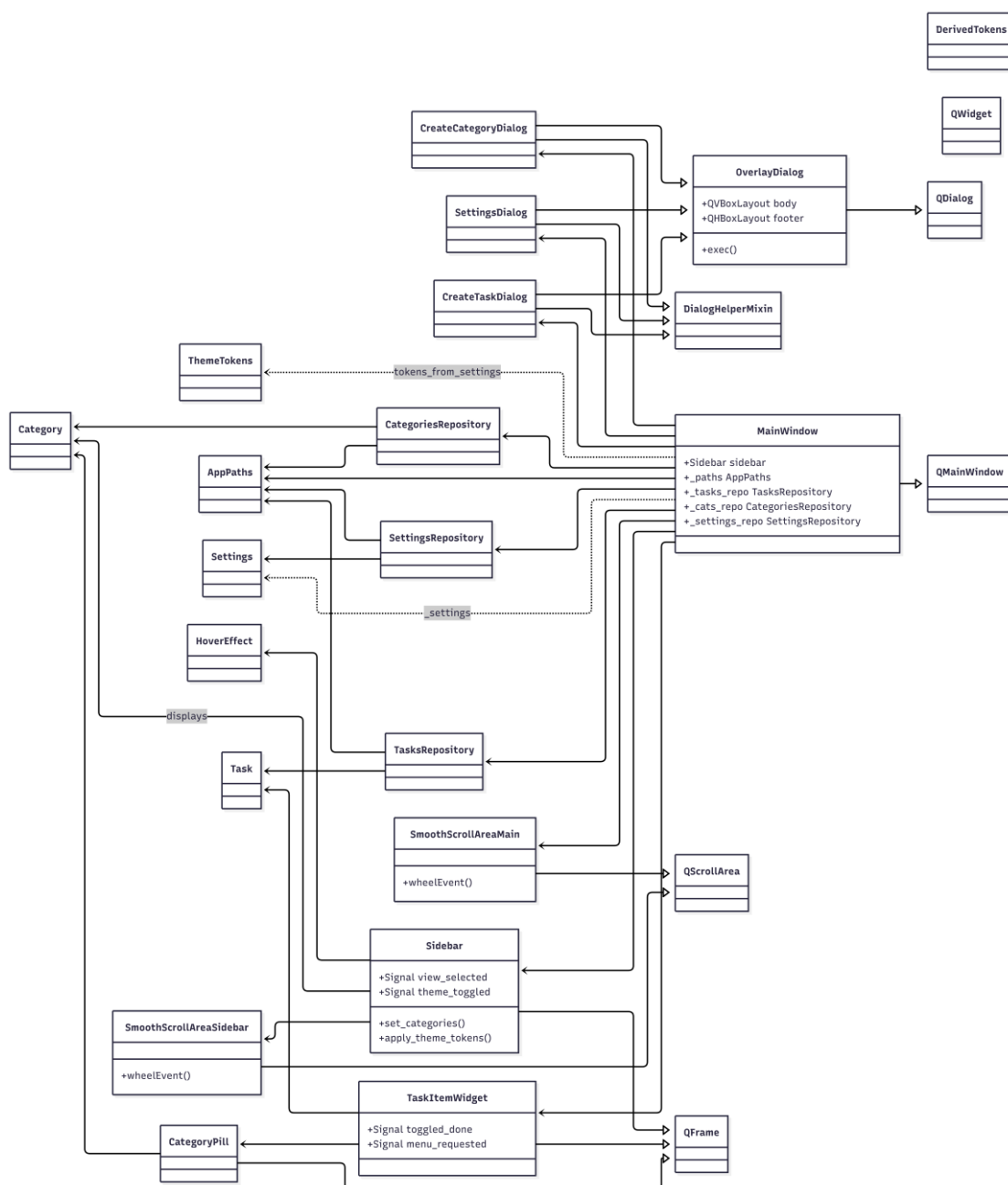


Рисунок 4.3.3 – Диаграмма классов ПО «Менеджер задач»

4.3.2 Разработка пользовательского интерфейса

Интерфейс приложения построен по принципу единого окна с динамической боковой панелью. Основные визуальные компоненты реализованы через стандартные и кастомные объекты PySide6:

- MainWindow (QMainWindow) – главное окно, содержащее область навигации и список задач;
- Sidebar (QFrame) – боковая панель для переключения категорий и фильтров. Реализована с эффектом плавного скрытия через QPropertyAnimation;
- TaskItemWidget (QFrame) – кастомный виджет для отображения отдельной задачи, включающий в себя QCheckBox для отметки выполнения и QLabel для текста и метаданных;
- OverlayDialog (QDialog) – базовый класс для модальных окон (создание задачи, настройки), реализующий эффект затемнения основного интерфейса.

Элементы управления стилизованы с помощью Qt Style Sheets (QSS). Для кнопок навигации (NavButton) и системных кнопок применен кастомный графический эффект HoverEffect, реализованный через QGraphicsColorizeEffect для визуального отклика при наведении курсора.

Экранные формы работы программного обеспечения приведены на рисунках 4.3.4 – 4.3.5.

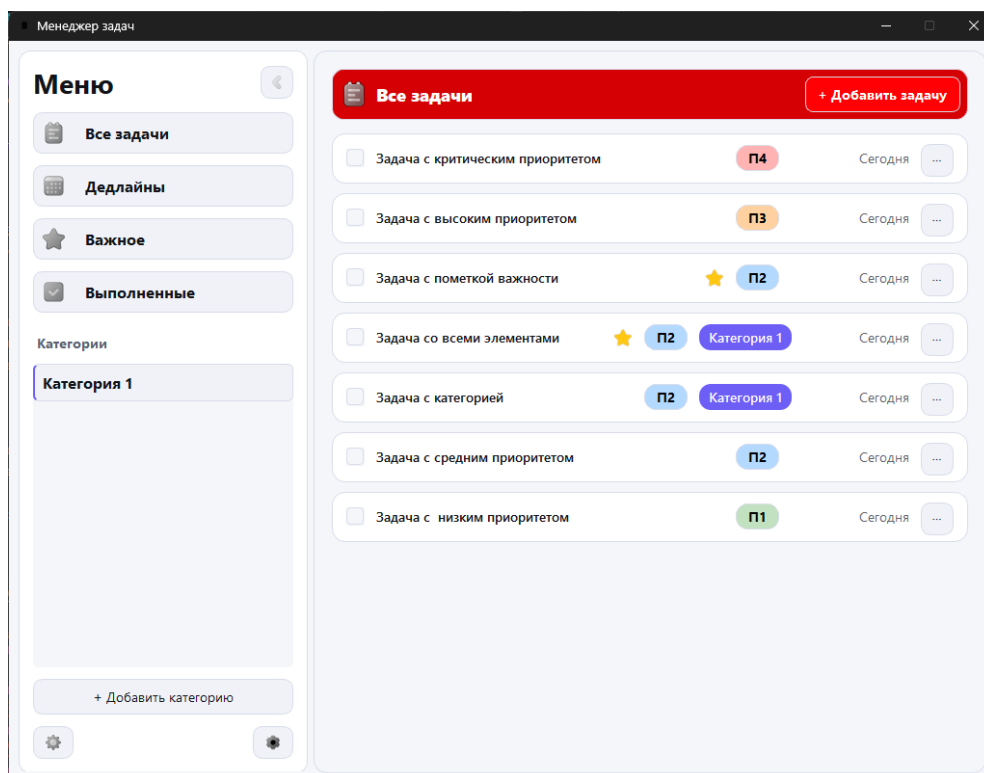


Рисунок 4.3.4 – Главное окно приложения

×

Создание задачи

Название задачи

Введите название...

Название должно кратко описывать суть

Выбор категории

Без категории

Выберите категорию, к которой относится задача

Дата дедлайна

04.04.2026

Поставьте отметку, к какому сроку задача должна быть выполнена

Приоритет

Средний

Выше приоритет — выше задача в списке

Пометка важности

☐
Важная задача

Используйте для задач, требующих повышенного внимания

Отменить

Создать

Рисунок 4.3.5 – Диалоговое окно создания новой задачи

Вывод по разделу:

Разработанное программное обеспечение обладает модульной структурой, что упрощает его поддержку и масштабирование. Использование JSON-хранилища обеспечивает переносимость данных, а применение PySide6 позволило создать современный и отзывчивый интерфейс.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. 20 причин освоить программирование на Python // SEOnews. URL: <https://www.seonews.ru/analytics/top-20-prichin-osvoit-programmirovanie-na-python-dlya-nachinayushchikh-i-detey/> (дата обращения: 13.05.2025).
2. TIOBE Index for January 2026 // TIOBE. URL: <https://www.tiobe.com/tiobe-index/> (дата обращения: 31.01.2026).
3. Developer Survey 2025 // Stack Overflow. URL: <https://survey.stackoverflow.co/2025/technology#most-popular-technologies-dev-envs> (дата обращения: .01.2026).
4. Телеграм-боты: какие бывают, как создать и как работать // Победа Digital. URL: <https://pbd.space/blog/telegram-boty-kakiye-byvayut-kak-sozdat-i-kak-rabotat/> (дата обращения: 13.05.2025).
5. Лутц, М. Изучаем Python : в 2 т. Т. 1 / М. Лутц ; пер. с англ. А. Д. Кошелева. – 5-е изд. – Санкт-Петербург : Диалектика, 2019. – 832 с.
6. Бизли, Д. Python. Подробный справочник / Д. Бизли ; пер. с англ. А. А. Слинкина. – 4-е изд. – Санкт-Петербург : Символ-Плюс, 2010. – 864 с.
7. Горелик, М. Высокопроизводительный Python: практическое программирование для профессионалов / М. Горелик, И. Озвальд ; пер. с англ. А. А. Слинкина. – 2-е изд. – Санкт-Петербург : Питер, 2021. – 480 с.
8. Слаткин, Б. Эффективный Python: 90 способов написать лучший код / Б. Слаткин ; пер. с англ. А. А. Слинкина. – 2-е изд. – Москва : Вильямс, 2021. – 400 с.
9. ГОСТ 34.003–90. *Информационная технология. Термины и определения.*
10. Управление задачами // Хабр Трек URL: <https://track.habr.com/frontend/skill/task-management> (дата обращения: 24.01.2026).

11. Архипенков, С. Я. Руководство командой разработчиков программного обеспечения. Прикладные мысли / С. Я. Архипенков. – Москва: Пресс-код, 2021. – 232 с.
12. Аллен, Д. Как привести дела в порядок: искусство продуктивности без стресса *Getting Things Done: The Art of Stress-Free Productivity* / Дэвид Аллен; пер. с англ. Ю. Константинова. – 10-е изд. – Москва: Манн, Иванов и Фербер, 2023. – 416 с.
13. Кови С. Р. Семь навыков высокоэффективных людей: Мощные инструменты развития личности / Стивен Р. Кови; пер. с англ. – 16-е изд. – Москва: Альпина Паблишер, 2022. – 396 с.
14. Таненбаум, Э. Современные операционные системы / Э. Таненбаум, Х. Бос. – 4-е изд. – Санкт-Петербург: Питер, 2019. – 1120 с.
15. Норман, Д. А. Дизайн привычных вещей / Дональд А. Норман; пер. с англ. – Москва: Манн, Иванов и Фербер, 2022. – 384 с.
16. Дорофеев, М. Джедайские техники. Как приручить своего вшивого обезьяноида, воспитать прокрастинатора и правильно тратить мыслетопливо / Максим Дорофеев. – Москва: Манн, Иванов и Фербер, 2023. – 352 с.
17. Друкер, П. Ф. Эффективный руководитель / Питер Ф. Друкер; пер. с англ. – Москва: Альпина Паблишер, 2021. – 240 с.

ПРИЛОЖЕНИЕ А

Исходный код

Точка входа:

```
import sys

import os

import platform


from PySide6.QtWidgets import QApplication
from PySide6.QtGui import QIcon
from src.ui.main_window import MainWindow


def setup_app(app: QApplication):
    icon_path = "resources/icons/app_icon.png"

    app.setWindowIcon(QIcon(icon_path))
    app.setApplicationName("task-manager")

    system = platform.system()

    if system == "Windows":
        import ctypes
        ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID(
            "task.manager.app"
        )

    elif system == "Linux":
        app.setDesktopFileName("task-manager")


def main() -> int:
    app = QApplication(sys.argv)
    setup_app(app)
```

```

window = MainWindow()

window.show()

return app.exec()

if __name__ == "__main__":
    raise SystemExit(main())

```

Модуль MainWindow:

```

from __future__ import annotations

"""Главное окно приложения с сайдбаром и списком задач."""

from datetime import date, datetime, timedelta, timezone

from PySide6.QtCore import QEasingCurve, QEvent, QObject, QPropertyAnimation, Qt,
QTimer

from PySide6.QtGui import QCursor, QPixmap, QIcon

from pathlib import Path

from PySide6.QtWidgets import (
    QDialog,
    QFrame,
    QHBoxLayout,
    QLabel,
    QMainWindow,
    QMenu,
    QMessageBox,
    QPushButton,
    QScrollArea,
    QVBoxLayout,
    QWidget,
)

```

```

from src.core.models import Category, Settings, Task

from src.core.paths import AppPaths

from src.core.repositories import CategoriesRepository, SettingsRepository,
TasksRepository

from src.core.backup import BackupError, export_all, import_all

from src.ui.dialogs.create_category_dialog import CreateCategoryDialog

from src.ui.dialogs.create_task_dialog import CreateTaskDialog

from src.ui.dialogs.settings_dialog import SettingsDialog

from src.theme.theme import tokens_from_settings

from src.ui.styles.app import app_qss, font_from_settings

from src.ui.widgets.sidebar import Sidebar

from src.ui.widgets.task_item import TaskItemWidget

from src.utils.buttons import HoverEffect

from src.utils.icons import icons


from PySide6.QtCore import QPropertyAnimation, QEasingCurve


class SmoothScrollArea(QScrollArea):
    def __init__(self):
        super().__init__()

        self._anim = QPropertyAnimation(self.verticalScrollBar(), b"value")
        self._anim.setDuration(400)
        self._anim.setEasingCurve(QEasingCurve.OutCubic)

    def wheelEvent(self, event):
        sb = self.verticalScrollBar()

        delta = event.angleDelta().y()

        # целевая позиция
        new_value = sb.value() - delta

```

```

new_value = max(sb.minimum(), min(sb.maximum(), new_value))

self._anim.stop()

self._anim.setStartValue(sb.value())

self._anim.setEndValue(new_value)

self._anim.start()

class MainWindow(QMainWindow):

    """Главное окно приложения.

    Отвечает за:

    - инициализацию репозитория и загрузку данных;
    - построение основного интерфейса (сайдбар + список задач);
    - обработку событий (infinite scroll, фильтры, открытие диалогов).
    """

    def __init__(self) -> None:
        super().__init__()

        self._paths = AppPaths()

        self._paths.ensure()

        self._settings_repo = SettingsRepository(self._paths)
        self._cats_repo = CategoriesRepository(self._paths)
        self._tasks_repo = TasksRepository(self._paths, page_size=25)

        self._settings = self._settings_repo.load()
        self._tokens = tokens_from_settings(self._settings)
        self._categories: list[Category] = self._cats_repo.load_all()
        self._tasks_repo.ensure_seed_data()

        self._active_view_key = "all"

        self._page_index = 0

        self._loading = False

```

```

self._view_tasks: list[Task] = []

self._build_ui()

self._apply_settings()

self._reload_sidebar()

self._reset_and_load_first_page()

def _build_ui(self) -> None:
    """Создаёт все виджеты главного окна и настраивает layout-ы."""
    self.setWindowTitle("Менеджер задач")
    self.setFixedSize(1024, 768)

    root = QWidget(self)
    self.setCentralWidget(root)
    layout = QHBoxLayout(root)
    layout.setContentsMargins(10, 10, 10, 10)
    # layout.setSpacing(10)

    self.sidebar = Sidebar(self._tokens, root)
    self.sidebar.pin_btn.toggled.connect(self._on_pin_toggled)
    self.sidebar.view_selected.connect(self._on_view_selected)
    self.sidebar.add_category_requested.connect(self._open_create_category)
    self.sidebar.category_delete_requested.connect(self._on_delete_category)
    self.sidebar.settings_requested.connect(self._open_settings)
    self.sidebar.theme_toggled.connect(self._on_theme_toggled)

    self._sidebar_container = QFrame(root)
    self._sidebar_container.setObjectName("SidebarContainer")
    s_l = QVBoxLayout(self._sidebar_container)
    s_l.setContentsMargins(0, 0, 0, 0)
    s_l.addWidget(self.sidebar)
    self._sidebar_container.setMaximumWidth(0) # hidden by default
    layout.addWidget(self._sidebar_container)

```



```

        self._sidebar_anim = QPropertyAnimation(self._sidebar_container,
b"maximumWidth", self)

        self._sidebar_anim.setDuration(180)

        self._sidebar_anim.setEasingCurve(QEasingCurve.OutCubic)


        self._main = QFrame(root)
        self._main.setObjectName("MainArea")
        self._main.setFixedHeight(750)

        # self._main.setStyleSheet('border: 1px solid #000000') # Debug: показывает все
границы элементов

        m = QVBoxLayout(self._main)
        m.setContentsMargins(18, 18, 18, 18)
        m.setSpacing(14)
        layout.addWidget(self._main, 1)


        # Оборачиваем header в QWidget
        header_widget = QWidget(self._main)
        header_widget.setObjectName("Header")
        #print(header_widget.styleSheet())


        # Создаём layout для header
        header_layout = QHBoxLayout(header_widget)
        header_layout.setContentsMargins(8, 8, 8, 8)
        header_layout.setSpacing(8)


        # Иконка вида
        self._view_icon = QLabel("", self)
        self._view_icon.setFixedWidth(30)
        self._view_icon.setFixedHeight(30)
        self._icon_header = QPixmap.icons['all_tasks'])
        self._view_icon.setPixmap(self._icon_header)
        self._view_icon.setStyleSheet("font-size: 30px;")
        self._view_icon.setScaledContents(True)

```

```

header_layout.addWidget(self._view_icon)

# Заголовок вида
self._view_title = QLabel("Все задачи", header_widget)
self._view_title.setObjectName("ViewTitleHeader")
self._view_icon.setAlignment(Qt.AlignmentFlag.AlignVCenter)
header_layout.addWidget(self._view_title)

# Раздвигаем содержимое
header_layout.addStretch(1)

# Кнопка "Добавить задачу"
self.add_task = QPushButton("+ Добавить задачу", header_widget)
self.hover_add_task = HoverEffect(self.add_task, tokens=self._tokens,
anim_duration=750)

self.add_task.setObjectName("PrimaryButton")
self.add_task.clicked.connect(self._open_create_task)
header_layout.addWidget(self.add_task)

# Добавляем header_widget в основной layout
m.addWidget(header_widget)

self._scroll = SmoothScrollArea()
self._scroll.setWidgetResizable(True)
self._scroll.setFrameShape(QFrame.NoFrame)

self._list_host = QWidget(self._scroll)
self._list_layout = QVBoxLayout(self._list_host)
self._list_layout.setContentsMargins(0, 0, 0, 0)
self._list_layout.setSpacing(10)

self._spinner = QLabel("Загружаем задачи...", self._list_host)
self._spinner.setAlignment(Qt.AlignCenter)

```

```

self._spinner.setObjectName("LoadingSpinner")

self._spinner.hide()

self._empty_state = QLabel("Задач не осталось, Вы хорошо постарались",
self._list_host)

self._empty_state.setAlignment(Qt.AlignCenter)

self._empty_state.setObjectName("EmptyState")

self._empty_state.hide()


self._list_layout.addWidget(self._empty_state)

self._list_layout.addWidget(self._spinner)

self._list_layout.addStretch(1)

self._scroll.setWidget(self._list_host)

m.addWidget(self._scroll, 1)


self._scroll.verticalScrollBar().valueChanged.connect(self._on_scroll)


# Hover-to-reveal region
# Important: we install the filter on the *application* so mouse move
# events from children still reach us (otherwise hover detection is flaky).
self.setMouseTracking(True)
root.setMouseTracking(True)
self._main.setMouseTracking(True)
self._scroll.viewport().setMouseTracking(True)
# Use QApplication.instance() without importing at top to keep imports lean.
from PySide6.QtWidgets import QApplication

QApplication.instance().installEventFilter(self)


self._pinned = False


def eventFilter(self, watched: QObject, event: QEvent) -> bool: # noqa: N802
    """Отслеживает движение мыши, чтобы показывать/скрывать сайдбар."""
    if not self.isVisible():

```

```

        return super().eventFilter(watched, event)

    if event.type() == QEvent.MouseMove and not self._pinned:
        # We need window-local coordinates regardless of which child emitted the
event.

        pos = self.mapFromGlobal(QCursor.pos())

        if 0 <= pos.x() <= 18:

            self._show_sidebar()

        elif pos.x() > self._sidebar_container.maximumWidth() + 20:

            self._hide_sidebar()

    return super().eventFilter(watched, event)

def _show_sidebar(self) -> None:
    """Запускает анимацию раскрытия сайдбара."""
    if self._sidebar_container.maximumWidth() >= 300:
        return

    self._sidebar_anim.stop()

    self._sidebar_anim.setStartValue(self._sidebar_container.maximumWidth())
    # self._sidebar_anim.setEasingCurve(QEasingCurve.Type.InSine) # Анимация лагает
    self._sidebar_anim.setEndValue(300)

    self._sidebar_anim.setDuration(650)

    self._sidebar_anim.start()

def _hide_sidebar(self) -> None:
    """Запускает анимацию скрытия сайдбара."""
    if self._sidebar_container.maximumWidth() <= 0:
        return

    self._sidebar_anim.stop()

    self._sidebar_anim.setStartValue(self._sidebar_container.maximumWidth())
    # self._sidebar_anim.setEasingCurve(QEasingCurve.Type.InSine) # Анимация лагает
    self._sidebar_anim.setEndValue(0)

    self._sidebar_anim.setDuration(650)

    self._sidebar_anim.start()

```

```

def _on_pin_toggled(self, checked: bool) -> None:
    """Фиксирует сайдбар в раскрытом состоянии или возвращает режим hover."""
    self._pinned = checked
    self.sidebar.pin_btn.setText("")
    self.sidebar.pin_btn.setIcon(QPixmap())
    if checked:
        self.sidebar.pin_btn.setIcon(QPixmap.icons['collapse_sidebar']))
        self._show_sidebar()
    else:
        self.sidebar.pin_btn.setIcon(QPixmap.icons['expand_sidebar']))
        self._hide_sidebar()

def _apply_settings(self) -> None:
    """Применяет текущие настройки темы/шрифта ко всему окну."""
    self._tokens = tokens_from_settings(self._settings)
    self.setStyleSheet(app_qss(self._tokens))
    self.sidebar.apply_theme_tokens(self._tokens)
    self.setFont(font_from_settings(self._settings))
    self.hover_add_task.set_tokens(self._tokens)
    self.sidebar.set_theme_checked(dark=self._settings.theme == "dark")

def _reload_sidebar(self) -> None:
    """Обновляет список категорий в сайдбаре."""
    self._categories = self._cats_repo.load_all()
    self.sidebar.set_categories(self._categories)

def _on_delete_category(self, category_id: str) -> None:
    """Удаляет категорию и все задачи в ней после подтверждения."""
    cat = next((x for x in self._categories if x.id == category_id), None)
    name = cat.name if cat else category_id
    result = QMessageBox.question(
        self,

```

```

        "Удалить категорию",
        f"Удалить категорию «{name}» и все задачи в ней? Это действие нельзя
отменить.",
        QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
        QMessageBox.StandardButton.No,
    )
    if result != QMessageBox.StandardButton.Yes:
        return

    self._tasks_repo.delete_all_by_category_id(category_id)
    self._cats_repo.delete_by_id(category_id)
    self._reload_sidebar()
    if self._active_view_key == f"category:{category_id}":
        self._on_view_selected("all")
    else:
        self._reset_and_load_first_page()

def _open_settings(self) -> None:
    """Открывает диалог настроек и сохраняет новые значения при подтверждении."""
    dlg = SettingsDialog(
        self,
        self._settings,
        on_export=self._export_backup,
        on_import=self._import_backup,
    )
    if dlg.exec() == QDialog.Accepted:
        self._settings = dlg.result_settings()
        self._settings_repo.save(self._settings)
        self._apply_settings()

def _export_backup(self, dest_path: str) -> None:
    try:
        export_all(self._paths, Path(dest_path))
    except Exception as e:

```

```

        QMessageBox.critical(self, "Экспорт", f"Не удалось экспортировать
данные.\n\n{e}")

        return

    QMessageBox.information(self, "Экспорт", "Экспорт успешно завершён.")

def _import_backup(self, src_path: str) -> None:
    result = QMessageBox.question(
        self,
        "Импорт",
        "Импорт заменит текущие настройки, категории и задачи. Продолжить?",
        QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
        QMessageBox.StandardButton.No,
    )
    if result != QMessageBox.StandardButton.Yes:
        return False

    try:
        import_all(self._paths, Path(src_path))
    except BackupError as e:
        QMessageBox.critical(self, "Импорт", f"Не удалось импортировать
данные.\n\n{e}")
        return False

    except Exception as e:
        QMessageBox.critical(self, "Импорт", f"Не удалось импортировать
данные.\n\n{e}")
        return False

    # Reload everything and refresh UI
    self._settings = self._settings_repo.load()
    self._apply_settings()
    self._reload_sidebar()
    self._reset_and_load_first_page()

    QMessageBox.information(self, "Импорт", "Импорт успешно завершён.")
    return True

```

```

def _on_theme_toggled(self, dark: bool) -> None:
    """Обрабатывает переключатель темы в сайдбаре (light/dark)."""
    self._settings = Settings(
        accent_color=self._settings.accent_color,
        theme="dark" if dark else "light",
        font_family=self._settings.font_family,
    )
    self._settings_repo.save(self._settings)
    self._apply_settings()

def _open_create_category(self) -> None:
    """Открывает диалог создания категории и добавляет её в JSON."""
    dlg = CreateCategoryDialog(self)
    if dlg.exec() != QDialog.Accepted:
        return

    name, color, icon_path = dlg.result()
    if not name:
        return

    self._cats_repo.add(name, color, icon_source_path=icon_path)
    self._reload_sidebar()
    self._reset_and_load_first_page()

def _open_create_task(self) -> None:
    """Открывает диалог создания задачи и сохраняет новую задачу."""
    dlg = CreateTaskDialog(self, categories=self._categories)
    if dlg.exec() != QDialog.Accepted:
        return

    title, cat_id, due, priority, important = dlg.result()
    if not title:
        return

    self._tasks_repo.add(
        title,
        category_id=cat_id,

```



```

        due=due,
        priority=priority,
        important=important,
    )

    self._reset_and_load_first_page()

def _on_view_selected(self, key: str) -> None:
    """Меняет текущий фильтр списка задач."""

    self._active_view_key = key

    # очистка перед установкой
    self._view_icon.setPixmap(QPixmap())
    self._view_icon.setText("")

    if key == "all":
        self._view_icon.setPixmap(QPixmap(icons['all_tasks']))
        self._view_title.setText("Все задачи")

    elif key == "deadlines":
        pixmap = QPixmap(icons['deadlines'])
        #print(pixmap.isNull()) # диагностика если нужно
        self._view_icon.setPixmap(pixmap)
        self._view_title.setText("Дедлайны")

    elif key == "important":
        pixmap = QPixmap(icons['important'])
        self._view_icon.setPixmap(pixmap)
        self._view_title.setText("Важное")

    elif key == "done":
        pixmap = QPixmap(icons['completed_tasks'])
        self._view_icon.setPixmap(pixmap)
        self._view_title.setText("Выполненные")

```

```

elif key.startswith("category:"):
    cid = key.split(":", 1)[1]
    c = next((x for x in self._categories if x.id == cid), None)
    self._view_icon.setText("📁")
    self._view_title.setText(c.name if c else "Категория")

self._reset_and_load_first_page()

def _view_sort_key(self, t: Task) -> tuple:
    # Completed view: newest completed first
    if self._active_view_key == "done":
        done_at = t.done_at
        if isinstance(done_at, datetime):
            # Ensure timestamp() is safe across platforms.
            if done_at.tzinfo is None:
                done_at = done_at.replace(tzinfo=timezone.utc)
            ts = done_at.timestamp()
        else:
            ts = 0.0
        return (-ts,)

    # Active views: higher priority first, then important, then earlier due dates
    due = t.due or date.max
    return (-int(t.priority or 0), -int(bool(t.important)), due.toordinal(),
t.title)

def _compute_view_tasks(self) -> list[Task]:
    tasks = self._tasks_repo.load_all()
    tasks = [t for t in tasks if self._passes_view_filter(t)]
    tasks.sort(key=self._view_sort_key)
    return tasks

```

```

def _reset_and_load_first_page(self) -> None:
    """Сбрасывает список и загружает первую страницу задач согласно фильтру."""
    self._page_index = 0
    self._clear_task_widgets()
    self._view_tasks = self._compute_view_tasks()
    if not self._view_tasks:
        self._empty_state.show()
        return
    self._load_next_page()

def _clear_task_widgets(self) -> None:
    """Удаляет все виджеты задач из layout-а и пересоздаёт строку со спиннером."""
    while self._list_layout.count() > 0:
        item = self._list_layout.takeAt(0)
        w = item.widget()
        if w is not None:
            w.deleteLater()
        else:
            # spacer/stretch
            pass

    self._spinner = QLabel("Загружаем задачи...", self._list_host)
    self._spinner.setAlignment(Qt.AlignCenter)
    self._spinner.setObjectName("LoadingSpinner")
    self._spinner.hide()

    self._empty_state = QLabel("Задач не осталось, Вы хорошо постарались",
self._list_host)

    self._empty_state.setAlignment(Qt.AlignCenter)
    self._empty_state.setObjectName("EmptyState")
    self._empty_state.hide()
    self._list_layout.addWidget(self._empty_state)
    self._list_layout.addWidget(self._spinner)
    self._list_layout.addStretch(1)

```

```

def _on_scroll(self) -> None:
    """Реагирует на прокрутку: при достижении низа подгружает следующую
    страницу."""

    if self._loading:
        return

    sb = self._scroll.verticalScrollBar()
    if sb.maximum() <= 0:
        return

    if sb.value() >= int(sb.maximum() * 0.85):
        self._load_next_page()

def _passes_view_filter(self, t: Task) -> bool:
    """Проверяет, подходит ли задача под текущий выбранный вид (фильтр)."""

    if self._active_view_key != "done" and t.done:
        return False

    if self._active_view_key == "done":
        return t.done

    if self._active_view_key == "all":
        return True

    if self._active_view_key == "important":
        return t.important

    if self._active_view_key == "deadlines":
        # Незавершённые: done уже отфильтрован выше. Дедлайн – просроченные или до
        # конца ближайших 7 дней.
        if t.due is None:
            return False

        horizon = date.today() + timedelta(days=7)
        return t.due <= horizon

    if self._active_view_key.startswith("category:"):
        cid = self._active_view_key.split(":", 1)[1]
        return t.category_id == cid

    return True

```

```

def _load_next_page(self) -> None:
    """Запускает асинхронную загрузку следующей страницы задач."""
    page_size = self._tasks_repo.page_size()
    if self._page_index * page_size >= len(self._view_tasks):
        return
    self._loading = True
    self._spinner.show()

    # simulate fetch delay so spinner is visible (and keeps UI smooth)
    QTimer.singleShot(250, self._finish_load_next_page)

def _finish_load_next_page(self) -> None:
    """Вызывается таймером: фактически подгружает задачи и добавляет их в
    список."""
    page_size = self._tasks_repo.page_size()
    start = self._page_index * page_size
    end = start + page_size
    page = self._view_tasks[start:end]
    self._page_index += 1
    self._spinner.hide()
    self._empty_state.hide()

    for t in page:
        cat = next((c for c in self._categories if c.id == t.category_id), None)
        w = TaskItemWidget(self._paths, t, category=cat, parent=self._list_host)
        w.toggled_done.connect(self._on_task_done_toggled)
        w.menu_requested.connect(self._on_task_menu_requested)
        # Insert above the spinner (spinner should stay at the bottom).
        spinner_index = self._list_layout.indexOf(self._spinner)
        if spinner_index < 0:
            spinner_index = max(0, self._list_layout.count() - 1)
        self._list_layout.insertWidget(spinner_index, w)

```

```

self._loading = False

def _on_task_done_toggled(self, task_id: str, done: bool) -> None:
    """Сохраняет изменение чекбокса выполнения задачи в репозиторий."""
    self._tasks_repo.set_done(task_id, done)
    self._reset_and_load_first_page()

def _on_task_menu_requested(self, task_id: str) -> None:
    """Открывает меню действий задачи (изменить/удалить)."""
    task = self._tasks_repo.get_by_id(task_id)

    if task is None:
        return

    menu = QMenu(self)

    toggle_done_action = menu.addAction("Отменить выполнение" if task.done else
    "Отметить выполненной")

    toggle_important_action = menu.addAction("Снять важность" if task.important
    else "Пометить важной")

    menu.addSeparator()

    edit_action = menu.addAction("Изменить")
    delete_action = menu.addAction("Удалить")

    chosen = menu.exec(QCursor.pos())

    if chosen == toggle_done_action:
        self._tasks_repo.set_done(task_id, not task.done)
        self._reset_and_load_first_page()
    elif chosen == toggle_important_action:
        self._tasks_repo.set_important(task_id, not task.important)
        self._reset_and_load_first_page()
    elif chosen == edit_action:
        self._edit_task(task_id)
    elif chosen == delete_action:
        self._delete_task(task_id)

def _edit_task(self, task_id: str) -> None:

```

```

        """Открывает диалог редактирования задачи."""
        task = self._tasks_repo.get_by_id(task_id)

        if task is None:
            return

        dlg = CreateTaskDialog(self, categories=self._categories, task=task)

        if dlg.exec() != QDialog.Accepted:
            return

        title, cat_id, due, priority, important = dlg.result()

        if not title:
            return

        self._tasks_repo.update(
            task_id,
            title=title,
            category_id=cat_id,
            due=due,
            priority=priority,
            important=important,
        )

        self._reset_and_load_first_page()

def _delete_task(self, task_id: str) -> None:
    """Удаляет задачу после подтверждения пользователем."""
    result = QMessageBox.question(
        self,
        "Удалить задачу",
        "Удалить выбранную задачу?",
        QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
        QMessageBox.StandardButton.No,
    )

    if result != QMessageBox.StandardButton.Yes:
        return

    self._tasks_repo.delete(task_id)

    self._reset_and_load_first_page()

```

Модуль сайдбара:

```
from __future__ import annotations

"""Левый сайдбар с навигацией по видам и категориям задач."""

from PySide6.QtCore import Qt, Signal
from PySide6.QtWidgets import (
    QFrame,
    QHBoxLayout,
    QLabel,
    QMenu,
    QPushButton,
    QScrollArea,
    QToolButton,
    QVBoxLayout,
    QWidget,
)

from PySide6.QtCore import QPropertyAnimation, QEasingCurve, QSize
from PySide6.QtGui import QPixmap
from src.core.models import Category
from src.utils.buttons import HoverEffect
from src.ui.styles.app import ThemeTokens

from src.utils.icons import icons
from src.core.paths import AppPaths
from src.core.repositories import SettingsRepository

class SmoothScrollArea(QScrollArea):
    def __init__(self):
        super().__init__()

        self._anim = QPropertyAnimation(self.verticalScrollBar(), b"value")
```



```

        self._anim.setDuration(400)

        self._anim.setEasingCurve(QEasingCurve.OutCubic)

def wheelEvent(self, event):
    sb = self.verticalScrollBar()

    delta = event.angleDelta().y()

    # целевая позиция
    new_value = sb.value() - delta

    new_value = max(sb.minimum(), min(sb.maximum(), new_value))

    self._anim.stop()
    self._anim.setStartValue(sb.value())
    self._anim.setEndValue(new_value)
    self._anim.start()

class Sidebar(QFrame):
    """Сайдбар со списком видов (все/дедлайны/важное) и пользовательских категорий."""

    view_selected = Signal(str) # view key: all, deadlines, important, category:<id>
    add_category_requested = Signal()
    category_delete_requested = Signal(str) # category id
    settings_requested = Signal()
    theme_toggled = Signal(bool) # True => dark

    def __init__(
        self,
        tokens: ThemeTokens,
        parent: QWidget | None = None,
    ) -> None:
        super().__init__(parent)
        self._tokens = tokens

```

```

self._paths = AppPaths()

self._cat_hovers: list[HoverEffect] = []

self.setObjectName("Sidebar")

self.setFixedHeight(750)

# screen_height = self.parent().height() # или
QApplication.primaryScreen().size().height()

self.setFixedWidth(300)

# self.setFixedHeight(int(screen_height)) # 60% от высоты экрана
# self.setStyleSheet("border-radius: 24px")


root = QVBoxLayout(self)

root.setContentsMargins(14, 14, 14, 14)

root.setSpacing(12)


top = QHBoxLayout()

top.setContentsMargins(0, 0, 0, 0)

title = QLabel("<b>Меню</b>", self)

title.setStyleSheet(""" \
    "font-size: 28px",

)

title.setObjectName("SidebarTitle")

top.addWidget(title)

top.addStretch(1)


self.pin_btn =QToolButton(self)

self._pin_hover = HoverEffect(self.pin_btn, tokens=tokens)

self.pin_btn.setCheckable(True)

self.pin_btn.setIcon(QPixmap.icons['expand_sidebar']))

self.pin_btn.setToolTip("Закрепить сайдбар")

self.pin_btn.setObjectName("PinButton")

self.pin_btn.setStyleSheet("""

```

```

        """)

    self.pin_btn.setFixedSize(34, 34)
    #self.pin_btn.clicked.connect(self.Sidebar._on_pin_toggled)
    top.addWidget(self.pin_btn)

    root.addLayout(top)

    self._nav_all = QPushButton("Все задачи", self)
    self._nav_all.setIcon(QPixmap(Icons['all_tasks']))
    self._nav_all.setIconSize(QSize(25, 25))
    # self._nav_all.set
    self._nav_all_hover = HoverEffect(self._nav_all, tokens=tokens)

    self._nav_deadlines = QPushButton("Дедлайны", self)
    self._nav_deadlines.setIcon(QPixmap(Icons['deadlines']))
    self._nav_deadlines_hover = HoverEffect(self._nav_deadlines, tokens=tokens)

    self._nav_important = QPushButton("Важное", self)
    self._nav_important.setIcon(QPixmap(Icons['important']))
    self._nav_important_hover = HoverEffect(self._nav_important, tokens=tokens)

    self._nav_done = QPushButton("Выполненные", self)
    # self._nav_done.set
    self._nav_done.setIcon(QPixmap(Icons['completed_tasks']))
    self._nav_done_hover = HoverEffect(self._nav_done, tokens=tokens)

    for b in (self._nav_all, self._nav_deadlines, self._nav_important,
self._nav_done):
        b.setObjectName("NavButton")
        b.setIconSize(QSize(25, 25))
        b.setText(" " + b.text())
        b.setCursor(Qt.PointingHandCursor)

```

```

        self._nav_all.clicked.connect(lambda: self.view_selected.emit("all"))

        self._nav_deadlines.clicked.connect(lambda:
self.view_selected.emit("deadlines"))

        self._nav_important.clicked.connect(lambda:
self.view_selected.emit("important"))

        self._nav_done.clicked.connect(lambda: self.view_selected.emit("done"))


root.addWidget(self._nav_all)
root.addWidget(self._nav_deadlines)
root.addWidget(self._nav_important)
root.addWidget(self._nav_done)


root.addSpacing(4)
cats_lbl = QLabel("Категории", self)
cats_lbl.setObjectName("SidebarSectionLabel")
root.addWidget(cats_lbl)


self._cats_container = QWidget()
self._cats_container.setAttribute(Qt.WA_StyledBackground, True)
self._cats_container.setStyleSheet("border-radius: 6px;")
self._cats_layout = QVBoxLayout(self._cats_container)
self._cats_container.setAutoFillBackground(True)
self._cats_layout.setContentsMargins(0, 0, 0, 0)
self._cats_layout.setSpacing(4)
self._cats_layout.addStretch(1)


self.scroll = SmoothScrollArea()
self.scroll.verticalScrollBar().valueChanged.connect(
lambda _: [w.update() for w in self._cats_container.findChildren(QWidget)]
)

self.scroll.setWidgetResizable(True)
self.scroll.setFrameShape(QFrame.NoFrame)
self.scroll.setWidget(self._cats_container)

```

```

root.addWidget(self.scroll, 1)

self.add_cat = QPushButton("+ Добавить категорию", self)
self._add_cat_hover = HoverEffect(self.add_cat, tokens=tokens)
self.add_cat.setObjectName("AddCategoryButton")
self.add_cat.clicked.connect(self.add_category_requested.emit)
root.addWidget(self.add_cat)

bottom = QHBoxLayout()
bottom.setContentsMargins(0, 0, 0, 0)

self._theme_btn = QToolButton(self)
self._theme_hover = HoverEffect(self._theme_btn, tokens=tokens)
self._theme_btn.setCheckable(True)
settings = SettingsRepository(self._paths).load()
if settings.theme == 'light':
    self._theme_btn.setIcon(
        QPixmap.icons['light_theme_indicator'])
    )
else:
    self._theme_btn.setIcon(
        QPixmap.icons['dark_theme_indicator'])
    )
self._theme_btn.setToolTip("Переключить тему")
self._theme_btn.setObjectName("ThemeButton")
self._theme_btn.setFixedSize(42, 34)
self._theme_btn.toggled.connect(self.theme_toggled.emit)
bottom.addWidget(self._theme_btn)

bottom.addStretch(1)

self.gear = QToolButton(self)
self._gear_hover = HoverEffect(self.gear, tokens=tokens)

```

```

self.gear.setIcon(QPixmap(icons['settings']))
self.gear.setToolTip("Настройки")
self.gear.setObjectName("SettingsButton")
self.gear.setFixedSize(42, 34)
self.gear.clicked.connect(self.settings_requested.emit)
bottom.addWidget(self.gear)

root.addLayout(bottom)

# Visual styling is applied globally from `src/ui/theme.py` QSS so it can
# react to theme changes (light/dark) consistently.

def set_theme_checked(self, *, dark: bool) -> None:
    """Программно выставляет состояние переключателя темы."""
    self._theme_btn.blockSignals(True)
    self._theme_btn.setChecked(dark)
    self._theme_btn.setIcon(QPixmap())
    if dark:
        self._theme_btn.setIcon(
            QPixmap(icons['dark_theme_indicator'])
        )
    else:
        self._theme_btn.setIcon(
            QPixmap(icons['light_theme_indicator'])
        )

    # self._theme_btn.setText("☾" if dark else "☀")
    self._theme_btn.blockSignals(False)

def set_categories(self, categories: list[Category]) -> None:
    """Обновляет список кнопок категорий на панели."""
    self._cat_hovers.clear()

    # удаляем старые кнопки (статический отступ/стретч пересоздадим ниже)
    while self._cats_layout.count() > 0:

```

```

        item = self._cats_layout.takeAt(0)

        w = item.widget()

        if w is not None:

            w.deleteLater()

        else:

            # spacer/stretch

            pass

    for c in categories:

        b = QPushButton(f"{c.name}", self._cats_container)

        b.setObjectName("NavButton")

        self._cat_hovers.append(HoverEffect(b, tokens=self._tokens))

        b.setStyleSheet(f"QPushButton#NavButton{{ border-left: 2px solid {c.color};
font-size: 16px}}")

        b.clicked.connect(lambda _=False, cid=c.id:
self.view_selected.emit(f"category:{cid}"))

        b.setContextMenuPolicy(Qt.ContextMenuPolicy.CustomContextMenu)

        b.customContextMenuRequested.connect(

            lambda pos, cid=c.id, btn=b:
self._on_category_context_menu(btn.mapToGlobal(pos), cid)

        )

        self._cats_layout.addWidget(b)

    self._cats_layout.addStretch(1)

def _on_category_context_menu(self, global_pos, category_id: str) -> None:

    menu = QMenu(self)

    delete_action = menu.addAction("Удалить категорию...")

    chosen = menu.exec(global_pos)

    if chosen == delete_action:

        self.category_delete_requested.emit(category_id)

def apply_theme_tokens(self, tokens: ThemeTokens) -> None:

```

```

self._tokens = tokens

self._pin_hover.set_tokens(tokens)

self._theme_hover.set_tokens(tokens)

self._nav_all_hover.set_tokens(tokens)

self._nav_deadlines_hover.set_tokens(tokens)

self._nav_important_hover.set_tokens(tokens)

self._nav_done_hover.set_tokens(tokens)

self._gear_hover.set_tokens(tokens)

self._add_cat_hover.set_tokens(tokens)

for h in self._cat_hovers:
    h.set_tokens(tokens)

```

Вспомогательный модуль для создания окна диалога:

```

from __future__ import annotations

```

```

"""Базовый диалог с затемняющим оверлеем поверх родительского окна."""

```

```

from PySide6.QtCore import Qt
from PySide6.QtWidgets import (
    QDialog,
    QFrame,
    QHBoxLayout,
    QLabel,
    QPushButton,
    QVBoxLayout,
    QWidget,
    QGraphicsDropShadowEffect,
)

```

```

class OverlayDialog(QDialog):

```

```

    """Фрейм-диалог."""

```

```

    def __init__(self, parent: QWidget, *, title: str) -> None:

```



```

super().__init__(parent)

# === Окно диалога ===
self.setWindowFlags(Qt.Dialog | Qt.FramelessWindowHint)
self.setModal(True)

# === Оверлей (затемнение) ===
self._overlay = QWidget(parent)
self._overlay.setObjectName("Overlay")
# self._overlay.setWindowFlags(Qt.FramelessWindowHint | Qt.Tool)
# self._overlay.setAttribute(Qt.WA_StyledBackground, True)

# === Карточка ===
self._card = QFrame(self)
self._card.setObjectName("DialogCard")
# self._card.setStyleSheet("""
# QFrame#DialogCard {
#     background: rgba(30, 30, 30, 0.95);
#     border-radius: 14px;
#     border: 1px solid rgba(255,255,255,0.06);
# }
# """)

# Тень
shadow = QGraphicsDropShadowEffect(self)
shadow.setBlurRadius(40)
shadow.setOffset(0, 10)
shadow.setColor(Qt.black)
self._card.setGraphicsEffect(shadow)

# корневой слой
self._root = QVBoxLayout(self)
self._root.setContentsMargins(0, 0, 0, 0)

```

```

self._root.addWidget(self._card)

# слой карточки
card_layout = QVBoxLayout(self._card)
card_layout.setContentsMargins(16, 16, 16, 16)
card_layout.setSpacing(12)

# Header
header = QHBoxLayout()
header.setContentsMargins(0, 0, 0, 0)

self._title = QLabel(title, self._card)
self._title.setObjectName("DialogTitle")
self._title.setStyleSheet("""
QLabel#DialogTitle {
    font-size: 16px;
    font-weight: 800;
}
""")
header.addWidget(self._title)
header.addStretch(1)

close = QPushButton("×", self._card)
close.setFixedSize(32, 32)
close.clicked.connect(self.reject)
close.setObjectName("CloseButton")
close.setStyleSheet("""
QPushButton#CloseButton {
    background: rgba(255,255,255,0.10);
    border: 1px solid rgba(0,0,0,0.10);
    border-radius: 6px;
    font-weight: 800;
}

```

```

QPushButton#CloseButton: hover {
    background: #F47174;
}
""")
header.addWidget(close)

card_layout.addLayout(header)

# Тело
self.body = QVBoxLayout()
self.body.setContentsMargins(0, 0, 0, 0)
self.body.setSpacing(10)
card_layout.addLayout(self.body)

# Footer
self.footer = QHBoxLayout()
self.footer.setContentsMargins(0, 8, 0, 0)
self.footer.setSpacing(10)
card_layout.addLayout(self.footer)

self.setMinimumWidth(520)

def exec(self) -> int: # type: ignore[override]
    """Показ с затемнением."""
    p = self.parentWidget()
    if p is not None:
        self._overlay.setGeometry(p.rect())
        self._overlay.show()
        self._overlay.raise_()

        self.raise_()
        self.activateWindow()
        self._center_in_parent()

```

```

    try:
        return super().exec()

    finally:
        self._overlay.hide() # НЕ удаляем

def _center_in_parent(self) -> None:
    p = self.parentWidget()

    if p is None:
        return

    self.adjustSize()

    x = (p.width() - self.width()) // 2
    y = (p.height() - self.height()) // 2

    self.move(max(0, x), max(0, y))

def mousePressEvent(self, event):
    """Закрытие при клике вне карточки."""

    if not self._card.geometry().contains(event.pos()):
        self.reject()

    super().mousePressEvent(event)

```

Модуль диалога создания категории:

```
from __future__ import annotations
```

```
"""Диалог создания новой категории задач."""
```

```
from pathlib import Path
```

```
from typing import Optional
```

```
from PySide6.QtCore import Qt
```

```
from PySide6.QtGui import QColor
```

```
from PySide6.QtWidgets import (
```

```

        QFileDialog,
        QHBoxLayout,
        QLabel,
        QLineEdit,
        QPushButton,
        QColorDialog,
        QFrame,
        QVBoxLayout,
        QSizePolicy
    )

from src.utils.dialog import DialogHelperMixin
from src.ui.widgets.overlay_dialog import OverlayDialog

class CreateCategoryDialog(OverlayDialog, DialogHelperMixin):
    """Диалог «Создание категории» с выбором цвета и иконки."""

    def __init__(self, parent) -> None:
        """Создаёт поля ввода названия, цвета и загрузки иконки."""
        super().__init__(parent, title="Создание категории")

        self._icon_path: Optional[Path] = None
        self._color = "#6D5EF6"

        self._name = QLineEdit(self)
        self._name.setPlaceholderText("Введите название...")
        self._name.setFixedWidth(150)

        self._accent_btn = QPushButton("Выбрать цвет", self)
        self._accent_btn.setCursor(Qt.PointingHandCursor)
        self._accent_btn.clicked.connect(self._pick_color)

        self._accent_preview = QPushButton("", self)
        self._accent_preview.setFixedSize(28, 28)

```

```

self._accent_preview.clicked.connect(self._pick_color)

self._accent_preview.setObjectName("AccentPreview")

self._set_color_preview()


icon_row = QHBoxLayout()

self._icon_lbl = QLabel("Файл не выбран", self)
self._icon_lbl.setFixedSize(150, 35)
self._icon_lbl.setObjectName("CategoryIconLabel")
# self._icon_lbl.setStyleSheet(
#     """
#     background: transparent;

#     border-radius: 8px;
#     """
# )

self._icon_btn = QPushButton("Загрузить файл", self)
self._icon_btn.clicked.connect(self._pick_icon)

icon_row.addWidget(self._icon_lbl)
icon_row.addWidget(self._icon_btn)


self.body.addLayout(
    self._row(
        "Название категории",
        "Название, которое будет отображаться рядом с задачей",
        control_widget=self._name
    )
)


self.body.addWidget(self._divider())


self.body.addLayout(
    self._row(
        "Акцентный цвет",

```

```

        "Выбор акцентного цвета категории",
        control_layout=self._accent_control()
    )
)

self.body.addWidget(self._divider())

self.body.addLayout(
    self._row(
        "Иконка категории",
        "Загрузите иконку, которая будет выражать категорию",
        control_layout=icon_row
    )
)

create = QPushButton("Создать", self)
create.setObjectName("AcceptButton")
create.clicked.connect(self.accept)
cancel = QPushButton("Отменить", self)
cancel.clicked.connect(self.reject)

self.footer.addStretch(1)
self.footer.addWidget(cancel)
self.footer.addWidget(create)

def _accent_control(self) -> QHBoxLayout:
    """Возвращает layout с кнопкой выбора цвета и кругом-превью."""
    h = QHBoxLayout()
    h.setContentsMargins(0, 0, 0, 0)
    h.setSpacing(10)
    h.addWidget(self._accent_btn)
    h.addWidget(self._accent_preview)
    return h

```

```

def _set_color_preview(self) -> None:
    """Обновляет внешний вид круглой кнопки-превью акцентного цвета."""
    self._accent_preview.setStyleSheet(
        f"""
        QPushButton {{
            background: {self._color};
        }}
        """
    )

def _pick_color(self) -> None:
    """Открывает цветовой диалог для выбора цвета категории."""
    c = QColorDialog.getColor(QColor(self._color), self, "Акцентный цвет")
    if not c.isValid():
        return
    self._color = c.name()
    self._set_color_preview()

def _pick_icon(self) -> None:
    """Открывает файловый диалог для выбора иконки категории."""
    file_path, _ = QFileDialog.getOpenFileName(
        self,
        "Выбор иконки",
        "",
        "Images (*.png *.jpg *.jpeg *.webp *.bmp)",
    )
    if not file_path:
        return
    self._icon_path = Path(file_path)
    self._icon_lbl.setText(f"{self._icon_path.name}")

def result(self) -> tuple[str, str, Optional[Path]]:

```



```

        """Возвращает введенные данные категории (имя, цвет, путь к иконке)."""
        return (self._name.text().strip(), self._color, self._icon_path)

```

Модуль диалога создания задач

```

from __future__ import annotations

"""Диалог создания новой задачи."""

from datetime import date
from typing import Optional

from PySide6.QtCore import QDate
from PySide6.QtWidgets import QCheckBox, QComboBox, QDateEdit, QLineEdit, QPushButton

from src.core.models import Category, Task
from src.ui.widgets.overlay_dialog import OverlayDialog
from src.utils.dialog import DialogHelperMixin

class CreateTaskDialog(OverlayDialog, DialogHelperMixin):
    """Диалог «Создание задачи» с полями названия, категории и дедлайна."""

    def __init__(self, parent, *, categories: list[Category], task: Task | None = None)
    -> None:
        """Наполняет комбобокс категориями и настраивает поля ввода."""
        super().__init__(parent, title="Создание задачи" if task is None else
        "Изменение задачи")

        self._categories = categories

        self._task = task

        self._name = QLineEdit(self)
        self._name.setPlaceholderText("Введите название...")
        self._name.setFixedWidth(150)

```

```

self._cat = QComboBox(self)

self._cat.addItem("Без категории", userData=None)

for c in categories:
    self._cat.addItem(c.name, userData=c.id)


self._due = QDateEdit(self)
self._due.setCalendarPopup(True)
self._due.setDisplayFormat("dd.MM.yyyy")
self._due.setDate(QDate.currentDate())


self._priority = QComboBox(self)
# Higher number => higher priority
self._priority.addItem("Низкий", userData=1)
self._priority.addItem("Средний", userData=2)
self._priority.addItem("Высокий", userData=3)
self._priority.addItem("Критический", userData=4)
self._priority.setCurrentIndex(1) # default: medium
self._important = QCheckBox("Важная задача", self)
self._important.setObjectName("ImportantCheckBox")
self._important.setToolTip("Помеченная задача отображается с индикатором
важности")


if task is not None:
    self._name.setText(task.title)

    if task.category_id is None:
        self._cat.setCurrentIndex(0)
    else:
        for i in range(self._cat.count()):
            if self._cat.itemData(i) == task.category_id:
                self._cat.setCurrentIndex(i)
                break

    if task.due is not None:
        self._due.setDate(QDate(task.due.year, task.due.month, task.due.day))

```

```

        for i in range(self._priority.count()):
            if int(self._priority.itemData(i) or 0) == int(task.priority):
                self._priority.setCurrentIndex(i)
                break

        self._important.setChecked(bool(task.important))

    self.body.addLayout(
        self._row(
            "Название задачи",
            "Название должно кратко описывать суть",
            control_widget=self._name
        )
    )

    self.body.addWidget(self._divider())

    self.body.addLayout(
        self._row(
            "Выбор категории",
            "Выберите категорию, к которой относится задача",
            control_widget=self._cat
        )
    )

    self.body.addWidget(self._divider())

    self.body.addLayout(
        self._row(
            "Дата дедлайна",
            "Поставьте отметку, к какому сроку задача должна быть выполнена",
            control_widget=self._due
        )
    )

    self.body.addWidget(self._divider())

    self.body.addLayout(
        self._row(
            "Приоритет",

```

```

        "Выше приоритет — выше задача в списке",
        control_widget=self._priority,
    )
)

self.body.addWidget(self._divider())
self.body.addLayout(
    self._row(
        "Пометка важности",
        "Используйте для задач, требующих повышенного внимания",
        control_widget=self._important,
    )
)

create = QPushButton("Создать" if task is None else "Сохранить", self)
create.setObjectName("AcceptButton")
create.clicked.connect(self.accept)
cancel = QPushButton("Отменить", self)
cancel.clicked.connect(self.reject)

self.footer.addStretch(1)
self.footer.addWidget(cancel)
self.footer.addWidget(create)

def result(self) -> tuple[str, Optional[str], Optional[date], int, bool]:
    """Возвращает введенные пользователем данные о задаче."""
    title = self._name.text().strip()
    cat_id = self._cat.currentData()
    qd = self._due.date()
    due = date(qd.year(), qd.month(), qd.day()) if qd.isValid() else None
    priority = int(self._priority.currentData() or 0)
    important = self._important.isChecked()
    return title, cat_id, due, priority, important

```

Модуль диалога настроек:

```
from __future__ import annotations

"""Диалог настроек приложения (акцент, тема, шрифт)."""

from PySide6.QtCore import Qt
from PySide6.QtGui import QColor
from PySide6.QtWidgets import (
    QComboBox,
    QColorDialog,
    QFileDialog,
    QFrame,
    QHBoxLayout,
    QLabel,
    QPushButton,
    QVBoxLayout,
)

from typing import Callable, Optional

from src.core.models import Settings
from src.ui.widgets.overlay_dialog import OverlayDialog
from src.utils.dialog import DialogHelperMixin

class SettingsDialog(OverlayDialog, DialogHelperMixin):
    """Диалог «Настройки» поверх главного окна."""

    def __init__(
        self,
        parent,
        settings: Settings,
```

```

*,
on_export: Optional[Callable[[str], None]] = None,
on_import: Optional[Callable[[str], bool]] = None,
) -> None:
    """Инициализирует элементы управления на основе текущих настроек."""
    super().__init__(parent, title="Настройки")
    self._settings = settings
    self._on_export = on_export
    self._on_import = on_import

    self._accent_btn = QPushButton("Выбрать цвет", self)
    self._accent_btn.setCursor(Qt.PointingHandCursor)
    self._accent_btn.clicked.connect(self._pick_accent)

    self._accent_preview = QPushButton("", self)
    self._accent_preview.setFixedSize(28, 28)
    self._accent_preview.setCursor(Qt.PointingHandCursor)
    self._accent_preview.clicked.connect(self._pick_accent)
    self._accent_preview.setObjectName("AccentPreview")

    self._theme = QComboBox(self)
    self._theme.addItems(["Светлая", "Тёмная"])

    self._font = QComboBox(self)
    self._font.addItems(["Ubuntu Sans", "Inter", "Roboto", "Segoe UI"])
    if settings.font_family:
        i = self._font.findText(settings.font_family)
        if i >= 0:
            self._font.setCurrentIndex(i)

    self.body.addLayout(
        self._row(
            "Акцентный цвет",

```

```

        "Выбор акцентного цвета, используемого в приложении",
        control_layout=self._accent_control(),
    )
)
self.body.addWidget(self._divider())
self.body.addLayout(
    self._row(
        "Основная тема",
        "Выбор основной цветовой схемы оформления Актива",
        control_widget=self._theme,
    )
)
self.body.addWidget(self._divider())
self.body.addLayout(
    self._row(
        "Основной шрифт",
        "Выбор основного шрифта интерфейса и текста в Актива",
        control_widget=self._font,
    )
)
)

self.body.addWidget(self._divider())

self._export_btn = QPushButton("Экспорт...", self)
self._export_btn.setCursor(Qt.PointingHandCursor)
self._export_btn.clicked.connect(self._export_clicked)
self._export_btn.setEnabled(self._on_export is not None)

self._import_btn = QPushButton("Импорт...", self)
self._import_btn.setCursor(Qt.PointingHandCursor)
self._import_btn.clicked.connect(self._import_clicked)
self._import_btn.setEnabled(self._on_import is not None)

```

```

self.body.addLayout(
    self._row(
        "Экспорт и импорт",
        "Сохранение и восстановление всех настроек и задач из файла",
        control_layout=self._export_import_controls(),
    )
)

self._apply_initial()

self._save = QPushButton("Сохранить", self)
self._save.setObjectName("AcceptButton")
self._save.clicked.connect(self.accept)
cancel = QPushButton("Отменить", self)
cancel.clicked.connect(self.reject)

self.footer.addStretch(1)
self.footer.addWidget(cancel)
self.footer.addWidget(self._save)

def _accent_control(self) -> QHBoxLayout:
    """Возвращает layout с кнопкой выбора цвета и кругом-превью."""
    h = QHBoxLayout()
    h.setContentsMargins(0, 0, 0, 0)
    h.setSpacing(10)
    h.addWidget(self._accent_btn)
    h.addWidget(self._accent_preview)
    return h

def _apply_initial(self) -> None:
    """Применяет стартовые значения акцентного цвета и темы."""
    self._set_accent_preview(self._settings.accent_color)
    self._theme.setCurrentIndex(1 if self._settings.theme == "dark" else 0)

```



```

def _set_accent_preview(self, hex_color: str) -> None:
    """Обновляет цвет круга-превью для акцентного цвета."""
    self._accent_preview.setStyleSheet(
        f"""
        QPushButton#AccentPreview {{
            background: {hex_color};
        }}
        """
    )

def _pick_accent(self) -> None:
    """Открывает цветовой диалог и обновляет текущий акцентный цвет."""
    c = QColorDialog.getColor(QColor(self._settings.accent_color), self, "Акцентный
цвет")
    #print(c.name())
    if not c.isValid():
        return
    self._settings = Settings(
        accent_color=c.name(),
        theme=self._settings.theme,
        font_family=self._settings.font_family,
    )
    self._set_accent_preview(self._settings.accent_color)

def result_settings(self) -> Settings:
    """Возвращает объект Settings, собранный из введённых пользователем
значений."""
    theme = "dark" if self._theme.currentText() == "Тёмная" else "light"
    return Settings(
        accent_color=self._settings.accent_color,
        theme=theme,
        font_family=self._font.currentText(),
    )

```

```

def _export_import_controls(self) -> QHBoxLayout:
    h = QHBoxLayout()
    h.setContentsMargins(0, 0, 0, 0)
    h.setSpacing(10)
    h.addWidget(self._export_btn)
    h.addWidget(self._import_btn)
    return h

def _export_clicked(self) -> None:
    if self._on_export is None:
        return

    path, _ = QFileDialog.getSaveFileName(
        self,
        "Экспорт данных",
        "task_manager_backup.zip",
        "Backup (*.zip)",
    )
    if not path:
        return
    self._on_export(path)

def _import_clicked(self) -> None:
    if self._on_import is None:
        return

    path, _ = QFileDialog.getOpenFileName(
        self,
        "Импорт данных",
        "",
        "Backup (*.zip)",
    )
    if not path:
        return

```

```

ok = self._on_import(path)

# Если импорт успешен, закрываем диалог, чтобы пользователь не мог
# "пересохранить" старые значения поверх импортированных.

if ok:

    self.reject()

```

Модуль элемента задачи:

```

from __future__ import annotations

"""Виджеты для отображения одной строки задачи и плашки категории."""

from datetime import date
from typing import Optional

from PySide6.QtCore import Qt, Signal, QPropertyAnimation, QEasingCurve
from PySide6.QtGui import QIcon, QPixmap, QPainter, QBrush
from PySide6.QtWidgets import (
    QCheckBox,
    QFrame,
    QHBoxLayout,
    QLabel,
    QPushButton,
    QSizePolicy,
    QWidget,
)

from src.utils.icons import icons
from src.core.models import Category, Task
from src.core.paths import AppPaths

def _human_due(d: Optional[date]) -> str:
    """Возвращает человекочитаемую дату дедлайна (Сегодня/Завтра/ДД.ММ.ГГГГ)."""
    if not d:

```

```

        return ""

today = date.today()

if d == today:
    return "Сегодня"

try:
    from datetime import timedelta

    if d == today + timedelta(days=1):
        return "Завтра"

except Exception:
    pass

return d.strftime("%d.%m.%Y")

class CategoryPill(QFrame):
    """Небольшой виджет-плашка с цветом и иконкой категории."""

    def __init__(self, paths: AppPaths, category: Category, parent: QWidget | None =
None) -> None:
        super().__init__(parent)

        self._paths = paths

        self._category = category

        self.setObjectName("CategoryPill")

        self.setStyleSheet(
            """
            background-color: rgba(255,255,255,0.1);
            """
        )

        self.setSizePolicy(QSizePolicy.Maximum, QSizePolicy.Fixed)

        layout = QHBoxLayout(self)

        layout.setContentsMargins(10, 4, 10, 4)

        layout.setSpacing(6)

```

```

self._icon = QLabel(self)
self._icon.setFixedSize(14, 14)
self._icon.setScaledContents(True)
layout.addWidget(self._icon)

self._text = QLabel(category.name, self)
self._text.setObjectName("CategoryPilllText")
layout.addWidget(self._text)

self._apply_icon()
self._apply_style()

def _apply_icon(self) -> None:
    """Пробует загрузить и показать иконку категории из директории icons_dir."""
    if not self._category.icon_filename:
        self._icon.hide()
        return

    p = self._paths.icons_dir / self._category.icon_filename
    if not p.exists():
        self._icon.hide()
        return

    pix = QPixmap(str(p))
    if pix.isNull():
        self._icon.hide()
        return

    # Размер иконки
    size = min(self._icon.width(), self._icon.height())
    pix = pix.scaled(
        size,
        size,
        Qt.AspectRatioMode.IgnoreAspectRatio,

```

```

        Qt.TransformationMode.SmoothTransformation
    )

    # Создаём круг
    mask = QPixmap(size, size)
    mask.fill(Qt.GlobalColor.transparent)
    painter = QPainter(mask)
    painter.setRenderHint(QPainter.RenderHint.Antialiasing)
    painter.setBrush(QBrush(Qt.GlobalColor.white))
    painter.drawEllipse(0, 0, size, size)
    painter.end()

    pix.setMask(mask.createMaskFromColor(Qt.GlobalColor.transparent,
Qt.MaskMode.MaskInColor))

    self._icon.setPixmap(pix)
    self._icon.show()

def _apply_style(self) -> None:
    """Применяет стили плашки (заливка цветом категории и белый текст)."""
    self.setStyleSheet(
        f"""
        QFrame#CategoryPill {{
            background: {self._category.color};
            border-radius: 12px;
        }}
        QLabel#CategoryPillText {{
            color: white;
            font-weight: 600;
        }}
        """
    )

```

```

class TaskItemWidget(QFrame):
    """Одна строка задачи в списке."""

    toggled_done = Signal(str, bool)
    menu_requested = Signal(str)

    def __init__(
        self,
        paths: AppPaths,
        task: Task,
        *,
        category: Category | None,
        parent: QWidget | None = None,
    ) -> None:
        super().__init__(parent)

        self._paths = paths
        self._task = task
        self.setObjectName("TaskItem")
        self.setFixedHeight(52)

        layout = QHBoxLayout(self)
        layout.setContentsMargins(14, 10, 14, 10)
        layout.setSpacing(12)

        self._check = QCheckBox(self)
        self._check.setFixedSize(19, 19)
        self._check.setChecked(task.done)
        self._check.setObjectName("TaskCheckBox")
        self._check.stateChanged.connect(self._on_done_changed)
        layout.addWidget(self._check, 0, Qt.AlignVCenter)

        self._title = QLabel(task.title, self)
        self._title.setObjectName("TaskTitle")

```

```

self._title.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Preferred)
layout.addWidget(self._title, 1, Qt.AlignVCenter)

self._important = QLabel("", self)
self._important_icon = QPixmap(
    icons['important_mask'] if task.important else ""
)
self._important.setPixmap(self._important_icon)
self._important.setScaledContents(True)
self._important.setObjectName("TaskImportant")
self._important.setFixedWidth(20)
self._important.setFixedHeight(20)
self._important.setAlignment(Qt.AlignCenter)
layout.addWidget(self._important, 0, Qt.AlignVCenter)

self._priority = QLabel(self)
self._priority.setObjectName("TaskPriority")
self._priority.setFixedWidth(45)
self._priority.setFixedHeight(27)
self._priority.setAlignment(Qt.AlignCenter)
self._priority.setProperty("priority", int(task.priority))
self._priority.style().unpolish(self._priority)
self._priority.style().polish(self._priority)
self._priority.setText(f"⏏{max(0, int(task.priority))}")
layout.addWidget(self._priority, 0, Qt.AlignVCenter)

self._pill: QWidget
if category:
    self._pill = CategoryPill(paths, category, self)
else:
    self._pill = QLabel("", self)
    self._pill.setFixedWidth(1)
layout.addWidget(self._pill, 0, Qt.AlignVCenter)

```



```

self._due = QLabel(_human_due(task.due), self)
self._due.setObjectName("TaskDue")
self._due.setFixedWidth(110)
self._due.setAlignment(Qt.AlignRight | Qt.AlignVCenter)
layout.addWidget(self._due, 0, Qt.AlignVCenter)

self._menu = QPushButton("...", self)
self._menu.setObjectName("MenuButton")
self._menu.setFixedSize(34, 34)
self._menu.clicked.connect(lambda: self.menu_requested.emit(self._task.id))
layout.addWidget(self._menu, 0, Qt.AlignVCenter)

# Styling is handled globally by the app QSS for theme consistency.
self._title.setStyleSheet("QLabel#TaskTitle { font-weight: 600; }")
self._menu.setStyleSheet(
    "QPushButton#MenuButton { border-radius: 10px; padding: 0px; font-size:
18px; }"
)
self._priority.setStyleSheet(
    "QLabel#TaskPriority { font-weight: 800; opacity: 0.8; }"
)
self._important.setStyleSheet(
    "QLabel#TaskImportant { font-size: 14px; }"
)

def _on_done_changed(self) -> None:
    """Излучает сигнал при изменении чекбокса выполнения задачи."""
    self.toggled_done.emit(self._task.id, self._check.isChecked())

```

Модуль репозиторов приложения:

```

from __future__ import annotations

"""Репозитории для работы с JSON-данными (настройки, категории, задачи)."""

```

```

import shutil

import uuid

from dataclasses import asdict

from datetime import date, datetime, timezone

from pathlib import Path

from typing import Iterable, Optional


from .json_io import read_json, write_json_atomic

from .models import Category, Settings, Task

from .paths import AppPaths


def _today() -> date:
    """Возвращает сегодняшнюю дату (выделено в функцию для удобства тестирования)."""
    return date.today()


def _task_to_json(task: Task) -> dict:
    """Преобразует объект Task в словарь, готовый к сериализации в JSON."""
    d = asdict(task)

    d["due"] = task.due.isoformat() if task.due else None

    d["done_at"] = task.done_at.isoformat() if task.done_at else None

    return d


def _task_from_json(d: dict) -> Task:
    """Создаёт объект Task из словаря, прочитанного из JSON."""
    due = d.get("due")
    done_at = d.get("done_at")

    return Task(
        id=str(d["id"]),
        title=str(d.get("title", "")),
        category_id=d.get("category_id"),
        due=date.fromisoformat(due) if due else None,
        priority=int(d.get("priority", 0) or 0),
    )

```

```

        important=bool(d.get("important", False)),
        done=bool(d.get("done", False)),
        done_at=datetime.fromisoformat(done_at) if done_at else None,
    )

def _category_to_json(cat: Category) -> dict:
    """Преобразует Category в словарь для записи в JSON."""
    return asdict(cat)

def _category_from_json(d: dict) -> Category:
    """Создаёт Category из словаря, прочитанного из JSON."""
    return Category(
        id=str(d["id"]),
        name=str(d.get("name", "")),
        color=str(d.get("color", "#888888")),
        icon_filename=d.get("icon_filename"),
    )

class SettingsRepository:
    """Репозиторий для чтения и записи настроек приложения в JSON."""

    def __init__(self, paths: AppPaths) -> None:
        self._paths = paths

    def load(self) -> Settings:
        """Читает настройки из файла и дополняет их дефолтными значениями."""
        raw = read_json(self._paths.settings_file, default={})
        return Settings(
            accent_color=str(raw.get("accent_color", Settings().accent_color)),
            theme=str(raw.get("theme", Settings().theme)),
            font_family=str(raw.get("font_family", Settings().font_family)),
        )

```

```

def save(self, s: Settings) -> None:
    """Сохраняет объект Settings в JSON-файл."""
    write_json_atomic(self._paths.settings_file, asdict(s))

class CategoriesRepository:
    """Репозиторий для управления списком категорий."""

    def __init__(self, paths: AppPaths) -> None:
        self._paths = paths

    def load_all(self) -> list[Category]:
        """Возвращает все сохранённые категории из JSON-файла."""
        raw = read_json(self._paths.categories_file, default=[])
        if not isinstance(raw, list):
            return []
        return [_category_from_json(x) for x in raw if isinstance(x, dict)]

    def save_all(self, cats: Iterable[Category]) -> None:
        """Перезаписывает JSON-файл полным списком переданных категорий."""
        write_json_atomic(self._paths.categories_file, [_category_to_json(c) for c in
cats])

    def add(self, name: str, color: str, *, icon_source_path: Optional[Path]) ->
Category:
        """Создаёт новую категорию и сохраняет её.

        Дополнительно, если указан путь к иконке, копирует файл в директорию
        `icons_dir` и сохраняет только имя файла в поле `icon_filename`.
        """
        cats = self.load_all()
        cat_id = uuid.uuid4().hex

        icon_filename: Optional[str] = None
        if icon_source_path:

```

```

        suffix = icon_source_path.suffix.lower() or ".png"
        icon_filename = f"{cat_id}-{suffix}"
        dest = self._paths.icons_dir / icon_filename
        self._paths.icons_dir.mkdir(parents=True, exist_ok=True)
        shutil.copy2(icon_source_path, dest)

        cat = Category(id=cat_id, name=name.strip(), color=color,
            icon_filename=icon_filename)

        cats.append(cat)

        self.save_all(cats)

        return cat

def delete_by_id(self, cat_id: str) -> bool:
    """Удаляет категорию по ID из JSON и файл иконки в icons_dir, если был."""
    cats = self.load_all()
    removed = next((c for c in cats if c.id == cat_id), None)
    if removed is None:
        return False
    remaining = [c for c in cats if c.id != cat_id]
    if removed.icon_filename:
        icon_path = self._paths.icons_dir / removed.icon_filename
        if icon_path.exists():
            icon_path.unlink()
    self.save_all(remaining)
    return True

class TasksRepository:
    """Репозиторий задач с постраничным хранением в JSON.

    Задачи лежат в нескольких файлах page_0001.json, page_0002.json и т.д.
    Это позволяет подгружать их по страницам (infinite scroll), а не
    загружать все задачи сразу в память.

    """

```

```

def __init__(self, paths: AppPaths, *, page_size: int = 25) -> None:
    self._paths = paths
    self._page_size = page_size

def page_size(self) -> int:
    return self._page_size

def _page_files(self) -> list[Path]:
    """Возвращает отсортированный список файлов страниц с задачами."""
    self._paths.tasks_pages_dir.mkdir(parents=True, exist_ok=True)
    files = sorted(self._paths.tasks_pages_dir.glob("page_*.json"))
    return files

def _next_page_path(self) -> Path:
    """Определяет путь к следующему файлу страницы."""
    files = self._page_files()
    if not files:
        return self._paths.tasks_pages_dir / "page_0001.json"
    last = files[-1].stem # page_0001
    try:
        n = int(last.split("_", 1)[1])
    except Exception:
        n = len(files)
    return self._paths.tasks_pages_dir / f"page_{n + 1:04d}.json"

def load_page(self, page_index: int) -> list[Task]:
    """Загружает одну страницу задач по индексу (0-based).

    Если индекс вне диапазона, возвращает пустой список.
    """
    files = self._page_files()
    if page_index < 0 or page_index >= len(files):

```

```

        return []

    raw = read_json(files[page_index], default=[])

    if not isinstance(raw, list):
        return []

    return [_task_from_json(x) for x in raw if isinstance(x, dict)]

def load_all(self) -> list[Task]:
    """Загружает все задачи из всех страниц (без сортировки)."""
    tasks: list[Task] = []
    for i in range(self.page_count()):
        tasks.extend(self.load_page(i))
    return tasks

def page_count(self) -> int:
    """Возвращает текущее количество файлов-страниц."""
    return len(self._page_files())

def add(
    self,
    title: str,
    *,
    category_id: Optional[str],
    due: Optional[date],
    priority: int = 0,
    important: bool = False,
) -> Task:
    """Создаёт новую задачу и добавляет её в последнюю страницу.

    Если последняя страница заполнена (page_size), создаётся новый файл.
    """
    task = Task(
        id=uuid.uuid4().hex,
        title=title.strip(),

```

```

        category_id=category_id,
        due=due,
        priority=int(priority),
        important=bool(important),
        done=False,
        done_at=None,
    )

files = self._page_files()
if not files:
    path = self._paths.tasks_pages_dir / "page_0001.json"
    write_json_atomic(path, [_task_to_json(task)])
    return task

last_path = files[-1]
raw = read_json(last_path, default=[])
if not isinstance(raw, list):
    raw = []

if len(raw) >= self._page_size:
    new_path = self._next_page_path()
    write_json_atomic(new_path, [_task_to_json(task)])
    return task

raw.append(_task_to_json(task))
write_json_atomic(last_path, raw)
return task

def ensure_seed_data(self) -> None:
    """Создаёт пару стартовых задач при первом запуске приложения."""
    if self.page_count() > 0:
        return

    self.add("Welcome to TaskManagerApp", category_id=None, due=_today())

```



```

self.add("Pin the sidebar to keep it open", category_id=None, due=None)

def set_done(self, task_id: str, done: bool) -> bool:
    """Обновляет флаг выполнения задачи в JSON без загрузки всех задач.

    Стратегия: последовательно сканируем все страницы, находим нужную
    задачу по ID, меняем поле `done` и перезаписываем этот один файл.

    Возвращает:
        True, если задача была найдена и обновлена, иначе False.
    """
    files = self._page_files()
    for path in files:
        raw = read_json(path, default=[])
        if not isinstance(raw, list):
            continue
        updated = False
        for item in raw:
            if not isinstance(item, dict):
                continue
            if str(item.get("id")) == task_id:
                item["done"] = bool(done)
                item["done_at"] = datetime.now(timezone.utc).isoformat() if done
            else None
                updated = True
                break
        if updated:
            write_json_atomic(path, raw)
            return True
    return False

def get_by_id(self, task_id: str) -> Optional[Task]:
    """Возвращает задачу по ID или None, если не найдена."""

```

```

files = self._page_files()

for path in files:
    raw = read_json(path, default=[])

    if not isinstance(raw, list):
        continue

    for item in raw:
        if not isinstance(item, dict):
            continue

        if str(item.get("id")) == task_id:
            return _task_from_json(item)

return None

def update(
    self,
    task_id: str,
    *,
    title: str,
    category_id: Optional[str],
    due: Optional[date],
    priority: int,
    important: bool,
) -> bool:
    """Обновляет поля задачи по ID."""

    files = self._page_files()

    for path in files:
        raw = read_json(path, default=[])

        if not isinstance(raw, list):
            continue

        updated = False

        for item in raw:
            if not isinstance(item, dict):
                continue

            if str(item.get("id")) == task_id:

```

```

        item["title"] = title.strip()
        item["category_id"] = category_id
        item["due"] = due.isoformat() if due else None
        item["priority"] = int(priority)
        item["important"] = bool(important)
        updated = True
        break
    if updated:
        write_json_atomic(path, raw)
        return True
    return False

def set_important(self, task_id: str, important: bool) -> bool:
    """Обновляет флаг важности задачи по ID."""
    files = self._page_files()
    for path in files:
        raw = read_json(path, default=[])
        if not isinstance(raw, list):
            continue
        updated = False
        for item in raw:
            if not isinstance(item, dict):
                continue
            if str(item.get("id")) == task_id:
                item["important"] = bool(important)
                updated = True
                break
        if updated:
            write_json_atomic(path, raw)
            return True
    return False

def delete(self, task_id: str) -> bool:

```

```

        """Удаляет задачу по ID."""
        files = self._page_files()
        for path in files:
            raw = read_json(path, default=[])
            if not isinstance(raw, list):
                continue

            before = len(raw)
            raw = [x for x in raw if not (isinstance(x, dict) and str(x.get("id")) ==
task_id)]

            if len(raw) != before:
                write_json_atomic(path, raw)
                return True

        return False

def delete_all_by_category_id(self, category_id: str) -> int:
    """Удаляет все задачи с указанной категорией со всех страниц. Возвращает число
    удалённых задач."""
    removed = 0
    for path in self._page_files():
        raw = read_json(path, default=[])
        if not isinstance(raw, list):
            continue

        new_raw: list = []
        for item in raw:
            if not isinstance(item, dict):
                new_raw.append(item)
                continue

            if str(item.get("category_id")) == category_id:
                removed += 1
                continue

            new_raw.append(item)

        if len(new_raw) != len(raw):
            write_json_atomic(path, new_raw)

    return removed

```

Модуль для работы с путями приложения:

```
from __future__ import annotations

"""Работа с путями к файлам конфигурации и данным приложения."""

from dataclasses import dataclass
from pathlib import Path

from platformdirs import user_config_dir

@dataclass(frozen=True)
class AppPaths:
    """Инкапсулирует все директории и файлы данных приложения.

    Использует библиотеку platformdirs, чтобы автоматически подобрать
    корректную директорию конфигурации под текущую ОС (Linux, Windows, macOS).
    """

    app_name: str = "TaskManagerApp"

    @property
    def config_dir(self) -> Path:
        """Корневая конфигурационная директория приложения в профиле пользователя."""
        return Path(user_config_dir(self.app_name))

    @property
    def data_dir(self) -> Path:
        """Поддиректория с данными (JSON-файлы, страницы задач)."""
        return self.config_dir / "data"

    @property
    def icons_dir(self) -> Path:
```

```

        """Поддиректория, в которую копируются пользовательские иконки категорий."""
        return self.config_dir / "icons"

@property
def tasks_pages_dir(self) -> Path:
    """Поддиректория со страницами задач (page_0001.json и т.д.)."""
    return self.data_dir / "tasks_pages"

@property
def settings_file(self) -> Path:
    """Путь к JSON-файлу с настройками приложения."""
    return self.data_dir / "settings.json"

@property
def categories_file(self) -> Path:
    """Путь к JSON-файлу со списком категорий."""
    return self.data_dir / "categories.json"

def ensure(self) -> None:
    """Создаёт все нужные директории, если они ещё не существуют."""
    self.config_dir.mkdir(parents=True, exist_ok=True)
    self.data_dir.mkdir(parents=True, exist_ok=True)
    self.icons_dir.mkdir(parents=True, exist_ok=True)
    self.tasks_pages_dir.mkdir(parents=True, exist_ok=True)

```

Модуль с основными моделями приложения:

```

from __future__ import annotations

```

"""Работа с путями к файлам конфигурации и данным приложения."""

```

from dataclasses import dataclass
from pathlib import Path

```

```

from platformdirs import user_config_dir

@dataclass(frozen=True)
class AppPaths:
    """Инкапсулирует все директории и файлы данных приложения.

    Использует библиотеку platformdirs, чтобы автоматически подобрать
    корректную директорию конфигурации под текущую ОС (Linux, Windows, macOS).
    """

    app_name: str = "TaskManagerApp"

    @property
    def config_dir(self) -> Path:
        """Корневая конфигурационная директория приложения в профиле пользователя."""
        return Path(user_config_dir(self.app_name))

    @property
    def data_dir(self) -> Path:
        """Поддиректория с данными (JSON-файлы, страницы задач)."""
        return self.config_dir / "data"

    @property
    def icons_dir(self) -> Path:
        """Поддиректория, в которую копируются пользовательские иконки категорий."""
        return self.config_dir / "icons"

    @property
    def tasks_pages_dir(self) -> Path:
        """Поддиректория со страницами задач (page_0001.json и т.д.)."""
        return self.data_dir / "tasks_pages"

    @property

```

```

def settings_file(self) -> Path:
    """Путь к JSON-файлу с настройками приложения."""
    return self.data_dir / "settings.json"

@property
def categories_file(self) -> Path:
    """Путь к JSON-файлу со списком категорий."""
    return self.data_dir / "categories.json"

def ensure(self) -> None:
    """Создаёт все нужные директории, если они ещё не существуют."""
    self.config_dir.mkdir(parents=True, exist_ok=True)
    self.data_dir.mkdir(parents=True, exist_ok=True)
    self.icons_dir.mkdir(parents=True, exist_ok=True)
    self.tasks_pages_dir.mkdir(parents=True, exist_ok=True)

```

Модуль чтения и записью:

```

from __future__ import annotations

"""Утилиты чтения и записи JSON-файлов."""

import json
from pathlib import Path
from typing import Any

def read_json(path: Path, *, default: Any) -> Any:
    """Безопасно читает JSON-файл.

    Если файл не существует или повреждён, возвращает переданный default.
    Это позволяет не падать при первой загрузке или при некорректном файле.
    """
    if not path.exists():

```



```

        return default

    try:
        with path.open("r", encoding="utf-8") as f:
            return json.load(f)
    except json.JSONDecodeError:
        return default

def write_json_atomic(path: Path, data: Any) -> None:
    """Записывает данные в JSON-файл атомарно.

    Сначала создаётся временный файл *.tmp, в который пишется JSON,
    затем он переименовывается в целевой путь. Так мы избегаем ситуаций,
    когда приложение внезапно завершается посреди записи и файл остаётся битым.
    """

    path.parent.mkdir(parents=True, exist_ok=True)
    tmp = path.with_suffix(path.suffix + ".tmp")
    with tmp.open("w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)
    tmp.replace(path)

```

Модуль резервного копирования:

```

from __future__ import annotations

"""Экспорт/импорт всех данных приложения (настройки, категории, задачи, иконки).

```

Формат экспорта: zip-архив со структурой:

- meta.json
- data/settings.json
- data/categories.json
- data/tasks_pages/page_0001.json ...
- icons/<files...>

```

"""

import json
import shutil
import tempfile
import zipfile

from datetime import datetime, timezone
from pathlib import Path
from typing import Iterable

from .paths import AppPaths

class BackupError(RuntimeError):
    pass

def _zip_write_file(zf: zipfile.ZipFile, *, src: Path, arcname: str) -> None:
    if not src.exists() or not src.is_file():
        return
    zf.write(src, arcname=arcname)

def _iter_files(dir_path: Path) -> Iterable[Path]:
    if not dir_path.exists() or not dir_path.is_dir():
        return []
    return [p for p in dir_path.rglob("*") if p.is_file()]

def export_all(paths: AppPaths, dest_zip: Path) -> None:
    """Экспортирует все данные приложения в zip-архив."""
    dest_zip = Path(dest_zip)
    dest_zip.parent.mkdir(parents=True, exist_ok=True)

    meta = {
        "format": "task-manager-backup",
        "version": 1,
    }

```

```

    "app_name": paths.app_name,
    "created_at": datetime.now(timezone.utc).isoformat(),
    "included": {
        "settings": True,
        "categories": True,
        "tasks_pages": True,
        "icons": True,
    },
}

tmp = dest_zip.with_suffix(dest_zip.suffix + ".tmp")
if tmp.exists():
    tmp.unlink()

with zipfile.ZipFile(tmp, "w", compression=zipfile.ZIP_DEFLATED) as zf:
    zf.writestr("meta.json", json.dumps(meta, ensure_ascii=False, indent=2))

    _zip_write_file(zf, src=paths.settings_file, arcname="data/settings.json")
    _zip_write_file(zf, src=paths.categories_file, arcname="data/categories.json")

    for p in sorted(paths.tasks_pages_dir.glob("page_*.json")):
        _zip_write_file(zf, src=p, arcname=f"data/tasks_pages/{p.name}")

    for p in sorted(_iter_files(paths.icons_dir)):
        rel = p.relative_to(paths.icons_dir).as_posix()
        _zip_write_file(zf, src=p, arcname=f"icons/{rel}")

tmp.replace(dest_zip)

def _is_safe_member(name: str) -> bool:
    # Prevent Zip Slip / path traversal.
    if not name or name.endswith("/"):
        return True

```

```

p = Path(name)

if p.is_absolute():
    return False

if any(part in ("..", "") for part in p.parts):
    return False

return True

def _extract_selected(zf: zipfile.ZipFile, *, members: list[str], dest_dir: Path) ->
None:
    dest_dir.mkdir(parents=True, exist_ok=True)

    for name in members:
        if not _is_safe_member(name):
            raise BackupError(f"Небезопасный путь в архиве: {name!r}")

        if name.endswith("/"):
            continue

        target = dest_dir / name
        target.parent.mkdir(parents=True, exist_ok=True)

        with zf.open(name, "r") as src, target.open("wb") as out:
            shutil.copyfileobj(src, out)

def import_all(paths: AppPaths, src_zip: Path) -> None:
    """Импортирует все данные из zip-архива, заменяя текущие данные приложения."""
    src_zip = Path(src_zip)

    if not src_zip.exists() or not src_zip.is_file():
        raise BackupError("Файл импорта не найден.")

    with zipfile.ZipFile(src_zip, "r") as zf:
        names = zf.namelist()

        if "meta.json" not in names:
            raise BackupError("Некорректный архив: отсутствует meta.json.")

        # meta validation (soft)

        try:

```

```

raw_meta = json.loads(zf.read("meta.json").decode("utf-8"))

except Exception as e:
    raise BackupError("Некорректный архив: meta.json повреждён.") from e

if not isinstance(raw_meta, dict) or raw_meta.get("format") != "task-manager-
backup":
    raise BackupError("Некорректный архив: неизвестный формат.")

wanted = [n for n in names if n.startswith(("data/", "icons/"))]

with tempfile.TemporaryDirectory(prefix="taskmgr_import_") as td:
    staging = Path(td)
    _extract_selected(zf, members=wanted, dest_dir=staging)

    new_data = staging / "data"
    new_icons = staging / "icons"
    if not new_data.exists():
        raise BackupError("Некорректный архив: отсутствует папка data/.")
    if not new_icons.exists():
        new_icons.mkdir(parents=True, exist_ok=True)

    # Ensure target dirs exist before swap.
    paths.ensure()

    # Swap directories atomically-ish (best effort).
    backup_root = paths.config_dir /
f"_backup_{datetime.now(timezone.utc).strftime('%Y%m%d_%H%M%S')}"
    backup_root.mkdir(parents=True, exist_ok=True)

    if paths.data_dir.exists():
        shutil.move(str(paths.data_dir), str(backup_root / "data"))
    if paths.icons_dir.exists():
        shutil.move(str(paths.icons_dir), str(backup_root / "icons"))

    shutil.copytree(new_data, paths.data_dir, dirs_exist_ok=True)

```

```
shutil.copytree(new_icons, paths.icons_dir, dirs_exist_ok=True)
```

Модуль формирования темы оформления:

```
from __future__ import annotations

"""Формирование темы оформления и QSS-стилей для всего приложения."""

from dataclasses import dataclass
from src.core.models import Settings

@dataclass(frozen=True)
class ThemeTokens:
    """Набор базовых цветов для текущей темы.

    Эти значения затем подставляются в QSS-строку для виджетов.
    """

    accent: str
    bg: str
    surface: str
    surface_2: str
    text: str
    text_muted: str
    border: str

@dataclass(frozen=True)
class DerivedTokens:
    """Значения, которые передаются вспомогательным генераторам QSS-стилей
    """

    primary_bg: str
```

```
primary_text: str
accent_text: str
```

```
def tokens_from_settings(s: Settings) -> ThemeTokens:
    """Создаёт набор цветовых токенов из объекта Settings."""
    if s.theme == "light":
        return ThemeTokens(
            accent=s.accent_color,
            bg="#F5F6FA",
            surface="#FFFFFF",
            surface_2="#F0F2F7",
            text="#111318",
            text_muted="#5B6270",
            border="#D9DDEA",
        )
    return ThemeTokens(
        accent=s.accent_color,
        bg="#0F1116",
        surface="#151824",
        surface_2="#1B2030",
        text="#E9ECF5",
        text_muted="#A7AFC3",
        border="#2A3147",
    )
```

Модуль сборки всех стилей:

```
from __future__ import annotations

"""Формирование темы оформления и QSS-стилей для всего приложения."""

from PySide6.QtGui import QFont
from PySide6.QtGui import QColor
```

```

from src.core.models import Settings
from src.theme.theme import ThemeTokens, DerivedTokens

from src.ui.styles.base import base_qss
from src.ui.styles.typography import typography_qss
from src.ui.styles.controls import controls_qss
from src.ui.styles.labels import labels_qss
from src.ui.styles.buttons import buttons_qss, tool_buttons_qss
from src.ui.styles.components import components_qss

def app_qss(tokens: ThemeTokens) -> str:
    primary_bg = QColor(tokens.accent).darker(120).name()
    primary_text = contrast_text_color(primary_bg)
    accent_text = contrast_text_color(tokens.accent)
    derived_tokens = DerivedTokens(primary_bg, primary_text, accent_text)
    """Генерирует общую QSS-строку для приложения на основе токенов темы."""
    return "".join([
        base_qss(tokens),
        typography_qss(tokens),
        buttons_qss(tokens, derived_tokens),
        tool_buttons_qss(tokens),
        labels_qss(tokens, derived_tokens),
        controls_qss(tokens, derived_tokens),
        components_qss(tokens, derived_tokens)
    ])

def font_from_settings(s: Settings) -> QFont:
    """Создаёт базовый QFont на основе настроек пользователя."""
    f = QFont(s.font_family)
    f.setPointSize(14) # T0-D0: Сделать чтобы менялось, в данный момент не работает
    return f

```



```

def contrast_text_color(bg_hex: str) -> str:
    """Помогает на основе фона сделать текст более контрастным

    Args:
        bg_hex (str): цвет фона в HEX-формате

    Returns:
        str: HEX цвет текста, белый(#FFFFFF) или черный(#000000)
    """

    c = QColor(bg_hex)

    brightness = (
        0.299 * c.red() +
        0.587 * c.green() +
        0.114 * c.blue()
    )

    return "#000000" if brightness > 186 else "#FFFFFF"

```

Модуль формирования базовых стилей:

```

from __future__ import annotations

"""Базовые QSS-стили для всего приложения."""

from dataclasses import dataclass

from PySide6.QtGui import QFont
from PySide6.QtGui import QColor
from src.core.models import Settings
from src.theme.theme import ThemeTokens

```

```

def base_qss(tokens: ThemeTokens):
    return (
        f"""
            QWidget {{
                background: {tokens.bg};
            }}

            * {{
                font-size: 14px;
                color: {tokens.text};
            }}
        """
    )

```

Модуль формирования стилей для текста:

```

from __future__ import annotations

"""Базовые QSS-стили типографии для всего приложения."""

from dataclasses import dataclass

from PySide6.QtGui import QFont
from PySide6.QtGui import QColor
from src.core.models import Settings
from src.theme.theme import ThemeTokens

def typography_qss(tokens: ThemeTokens) -> str:
    return (
        f"""
            QLabel {{
                background: transparent;

```

```

        font-weight: 400;

    }}

    """

)

```

Модуль формирования стилей для кнопок:

```

from __future__ import annotations

"""Формирование QSS-стилей для отдельных элементов Кнопок."""

from dataclasses import dataclass

from PySide6.QtGui import QFont
from PySide6.QtGui import QColor
from src.core.models import Settings
from src.theme.theme import ThemeTokens, DerivedTokens

def buttons_qss(tokens: ThemeTokens, der_tokens: DerivedTokens) -> str:
    return (
        f"""
        QPushButton {{ /* Все кнопки */
            background: {tokens.surface_2};
            border: 1px solid {tokens.border};
            border-radius: 10px;
            padding: 8px 12px;
        }}
        QPushButton:hover {{
            background: {tokens.surface};
        }}
        QPushButton#NavButton {{ /* Все кнопки */
            background: {tokens.surface_2};
            border: 1px solid {tokens.border};
            border-radius: 10px;

```

```

        padding: 8px 8px;

        text-align: left;

        font-size: 16px;

        font-weight: 700;
    }}

QPushButton#NavButton:hover {{
    background: {tokens.surface};
}}

QPushButton#PrimaryButton {{ /* Кнопка добавления задачи */
    background: {tokens.accent};

    border: 0.6px solid {tokens.border};

    border-radius: 8px;

    color: {der_tokens.accent_text};

    font-weight: 800;
}}

QPushButton#PrimaryButton:hover {{
    border: 0.6px solid {tokens.border};

    border-radius: 8px;

    color: {der_tokens.accent_text};

    font-weight: 800;
}}

*/
QPushButton#AcceptButton {{ /* Кнопка подтверждения настроек, категории, задачи

    background: {der_tokens.primary_bg};

    border: 0.6px solid {tokens.border};

    border-radius: 8px;

    color: {der_tokens.primary_text};

    font-weight: 800;
}}

QPushButton#AcceptButton:hover {{
    background: {tokens.accent};

    border: 0.6px solid {tokens.border};

```

```

        border-radius: 8px;

        color: {der_tokens.primary_text};

        font-weight: 800;
    }}

    QPushButton#AccentPreview {{ /* Кнопка "круга" в категории и настройках */

        border: 2px solid {tokens.border};

        border-radius: 14px;

    }}

    """"

)

def tool_buttons_qss(tokens: ThemeTokens) -> str:
    return (
        f"""

        QToolButton#PinButton:hover, QToolButton#ThemeButton:hover,
        QToolButton#SettingsButton:hover {{

            background-color: rgba(0, 0, 0, 30); /* слегка темнеет */

            border-radius: 8px; /* скругление если нужно */

        }}

        QToolButton#PinButton, QToolButton#ThemeButton, QToolButton#SettingsButton {{

            background: {tokens.surface_2};

            border: 1px solid {tokens.border};

            border-radius: 10px;

        }}

        """"

    )

```

Модуль формирования стилей для текста:

```

from __future__ import annotations

""""Формирование QSS-стилей для отдельных элементов QLabel.""""

```

```

from dataclasses import dataclass

from PySide6.QtGui import QFont
from PySide6.QtGui import QColor
from src.core.models import Settings
from src.theme.theme import ThemeTokens, DerivedTokens

def labels_qss(tokens: ThemeTokens, der_tokens: DerivedTokens) -> str:
    return (
        f"""
        QLabel#TaskPriority {{ { /* "Чипс" приоритета */
            border: 1px solid {tokens.border};
            border-radius: 12px;
            color: black;
        }}

        QLabel#TaskPriority[priority="1"] {{ { /* Приоритет "низкий" */
            background: #C1E1C1;
        }}

        QLabel#TaskPriority[priority="2"] {{ { /* Приоритет "Средний" */
            background: #B3D9FF;
        }}

        QLabel#TaskPriority[priority="3"] {{ { /* Приоритет "Высокий" */
            background: #FFD1A1;
        }}

        QLabel#TaskPriority[priority="4"] {{ { /* Приоритет "Критический" */
            background: #FFB3B3;
        }}
        """
    )

```

```

        QLabel#CategoryIconLabel {{ /* "Чипс" с иконкой категории и кратким
описанием */

        background: transparent;

        border: 1px solid {tokens.border};

        border-radius: 8px;

    }}

    QLabel#ViewTitleHeader {{ /* "Отображение названия категории" */

        font-size: 18px;

        font-weight: 900;

        color: {der_tokens.accent_text};

    }}

    QLabel#TaskDue {{ /* Дедлайн */

        color: {tokens.text_muted};

    }}

    QLabel#SidebarSectionLabel {{ /* Подпись "Категории" перед фреймом
категорий*/

        color: {tokens.text_muted};

        font-weight: 700;

        padding-top: 6px;

    }}

    QLabel#LoadingSpinner {{ /* Спиннер "Загружаем задачи" */

        color: {tokens.text_muted};

        opacity: 0.7;

        padding: 10px;

    }}

    QLabel#EmptyState {{ /* Отображается, когда не осталось задач */

        color: {tokens.text_muted};

        background: transparent;

        border: 1px dashed {tokens.border};

```

```

        border-radius: 12px;

        padding: 14px;

        font-weight: 700;
    }}

    """

)

```

Модуль формирования стилей для контроллеров:

```

from __future__ import annotations

"""Формирование QSS-стилей для отдельных
элементов управления"""

from dataclasses import dataclass

from PySide6.QtGui import QFont
from PySide6.QtGui import QColor
from src.core.models import Settings
from src.theme.theme import ThemeTokens, DerivedTokens
from src.utils.icons import icons

def controls_qss(tokens: ThemeTokens, der_tokens: DerivedTokens) -> str:
    return (
        f"""
        QCheckBox#ImportantCheckBox {{
            background: transparent;

            border: 1px solid {tokens.border};

            border-radius: 10px;

            padding: 8px 10px;

        }}

        QCheckBox#ImportantCheckBox::indicator {{

```



```

        background: transparent;

        border: 1px solid {tokens.border};

        border-radius: 4px; /* ← скругление */

        width: 16px;

        height: 16px;
    }}

    QCheckBox#ImportantCheckBox::indicator:checked {{

        background: {der_tokens.primary_bg};

        border: 1px solid {tokens.border};

        image: url("resources/icons/check.png");

        border-radius: 4px;
    }}

    QCheckBox#TaskCheckBox::indicator {{

        background: transparent;

        width: 16px;

        height: 16px;

        border-radius: 4px;

        border: 1px solid {tokens.border};
    }}

    QCheckBox#TaskCheckBox::indicator:checked {{

        width: 16px;

        height: 16px;

        background-color: {der_tokens.primary_bg};

        border: 1px solid {tokens.border};

        border-radius: 4px;

        image: url("resources/icons/check.png");
    }}

    QLineEdit, QComboBox, QDateEdit {{

        background: {tokens.surface};

        border: 1px solid {tokens.border};

        border-radius: 10px;
    }}

```

```

        padding: 8px 10px;

        selection-background-color: {tokens.accent};
    }}

    QComboBox::drop-down {{
        border: 0px;

        width: 26px;
    }}

    QScrollArea {{
        border: none;

        background: transparent;
    }}

    QScrollBar:vertical {{
        background: transparent;

        width: 8px;

        margin: 1px;
    }}

    QScrollBar::handle:vertical {{
        background: {tokens.border};

        border-radius: 2px;

        min-height: 30px;
    }}

    QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical {{
        height: 0px;
    }}

    """"
)

```

Модуль формирования стилей для больших компонентов:

```
from __future__ import annotations
```

```

"""Формирование QSS-стилей для отдельных больших элементов, будь то HEADER и др"""

from dataclasses import dataclass

from PySide6.QtGui import QFont
from PySide6.QtGui import QColor
from src.core.models import Settings
from src.theme.theme import ThemeTokens, DerivedTokens

def components_qss(tokens: ThemeTokens, der_tokens: DerivedTokens) -> str:
    return (
        f"""
        QFrame#MainArea {{ /* Главное окно (ROOT) */
            border: 1px solid {tokens.border};
            border-radius: 14px;
        }}

        QWidget#Header {{ /* Верхняя часть у главного экрана (HEADER) */
            background-color: {der_tokens.primary_bg};
            border-radius: 12px;
            border: 1px solid {tokens.border};
        }}

        QFrame#TaskItem {{ /* Элемент "Задача" 1 строка (task 1 row) */
            background: {tokens.surface};
            border: 1px solid {tokens.border};
            border-radius: 14px;
        }}

        QFrame#Sidebar {{ /* Сайдбар (Sidebar) */
            background: {tokens.surface};
            border: 1px solid {tokens.border};
            border-radius: 12px;
        }}
        """
    )

```

```

}}

QWidget#Overlay {{ /* Диалоговые окна */
    background: rgba(10, 10, 10, 140);
}}

QFrame#DialogCard {{
    background: {tokens.surface};
    border: 1px solid {tokens.border};
    border-radius: 8px;
}}

QMenu {{ /* Выпадающее меню */
    background: {tokens.surface};
    border: 1px solid {tokens.border};
    border-radius: 10px;
    padding: 6px;
}}

QMenu::item {{ /* Элемент выпадающего меню */
    background: transparent;
    border-radius: 8px;
    padding: 8px 12px;
}}

QMenu::item:selected {{
    background: {tokens.surface_2};
    border-radius: 8px;
    padding: 8px 12px;
}}

""""

)

```

